Ngu Son Retreat Management System

Software Design Document

Table of Contents

1. Introduction

1.1 Purpose

The **Ngu Son Retreat Management System** is a comprehensive software platform designed specifically for the unique operational workflows of modern Wellness Resorts. Unlike standard Property Management Systems (PMS), this system integrates and centralizes complex logistical operations, including Retreat Package bookings, automated Spa and therapy scheduling, personalized dietary (F&B) management, and consolidated billing. The system acts as a secure hub that strictly protects sensitive guest health and allergy data while seamlessly integrating with external APIs for payment processing, calendar synchronization, and authentication.

Purpose Details:

- **Technical Blueprint:** Translate the functional and non-functional requirements defined in the SRS (Software Requirements Specification) into a detailed technical architecture tailored for the system's 5 core modules (Authentication & Health Profile, Booking, Spa Scheduling Engine, Dietary F&B, and Consolidated Checkout).

- **Implementation Guide:** Serve as a comprehensive guide for the development team to implement the system, ensuring strict adherence to critical business rules such as 2-dimensional double-booking prevention, Role-Based Access Control (RBAC), and data minimization principles.
- **Detailed Content:** Describe the multi-tier system architecture, database schema (ERD) with a focus on data encryption (ensuring compliance with Decree 356/2025 on personal data protection), sequence diagrams, and third-party API integration specifications (Payment Gateways, Google Calendar, SSO).
- **Target Audience:**
    - *Development Team:* To understand structural components, coding standards, and data privacy enforcement across all interconnected modules.
    - *Testing Team:* To design integration and system test cases based on specific technical logic (e.g., verifying consolidated billing constraints prior to check-out).
    - *Project Stakeholders:* To review the technical feasibility, domain compliance (such as AHLEI and GWI standards), and the overall scope of the proposed solution.

1.2 Definitions, Acronyms and Abbreviations

Term	Definition	Context in Ngu Son Retreat Project
NSR	Ngu Son Retreat	The overall software project name for the Wellness Resort & Spa Management System.

PMS	Property Management System	Traditional hotel software. NSR extends standard PMS capabilities to manage bundled retreat experiences, dietary profiles, and complex spa scheduling.
F&B	Food and Beverage	Refers to the department and system module handling daily dietary management, personalized meal prep, and extra a-la-carte orders.
Folio	Guest Folio	The central accounting mechanism (per AHLEI standards) where all point-of-sale debts (Spa, F&B) are aggregated for the final consolidated check-out invoice.
RBAC	Role-Based Access Control	A security mechanism enforced in the backend to limit data access strictly based on staff roles (e.g., masking health data from Chefs, showing it only to Therapists).
SSO	Single Sign-On	An authentication API (Google Identity / Facebook Login) used for seamless and secure guest registration.
GWI	Global Wellness Institute	The organization providing the industry-standard taxonomy used to categorize and set up Master Data for "Retreat Packages".
AHLEI	American Hotel & Lodging Educational Institute	The organization defining the standard operational and accounting logic used for the Guest Folio and billing constraints.

ERD	Entity Relationship Diagram	A graphical representation of the database schema, detailing the relationships between the 5 modules and identifying encrypted sensitive data fields.
API	Application Programming Interface	External third-party tools integrated into the system for Payments (Stripe/VNPay), scheduling (Google Calendar), and notifications (SendGrid).
CRUD	Create, Read, Update, Delete	The four basic operations of persistent storage, heavily utilized by the System Manager (Admin) to manage Master Data (Villas, Packages, Spa Services, Staff).
DAO	Data Access Object	An architectural pattern used by the development team to separate the data persistence logic from the business logic in the backend controllers.

1.3 References

1.3.1 Software Development Project Specification

- **Project Title:** Ngu Son Retreat - A Wellness Resort & Spa Management System
- **Document Code:** SWP391-HOS-03
- **Description:** This is the primary source of requirements, describing all use cases, actors, module assignments, and strict business rules for the system.

1.3.2 Legal & Data Protection Regulations

- **Vietnam Decree 356/2025/ND-CP on Personal Data Protection:** Effective Jan 1, 2026. Referenced for Article 4 (Sensitive Data - Health & Biometrics) and Article 6 (Explicit Consent) to implement database encryption and UI consent boxes.
- **Law on Residence 2020 (Luật Cư trú 2020):** Referenced for mandatory temporary residence declaration rules applied to the Check-in process and police reporting.

1.3.3 Hospitality & Wellness Domain Standards

- **Global Wellness Institute (GWI) Standards:** Taxonomy of Wellness Tourism, used as the baseline for configuring Master Data for "Retreat Packages" (e.g., Mindfulness Retreat, Detoxification).

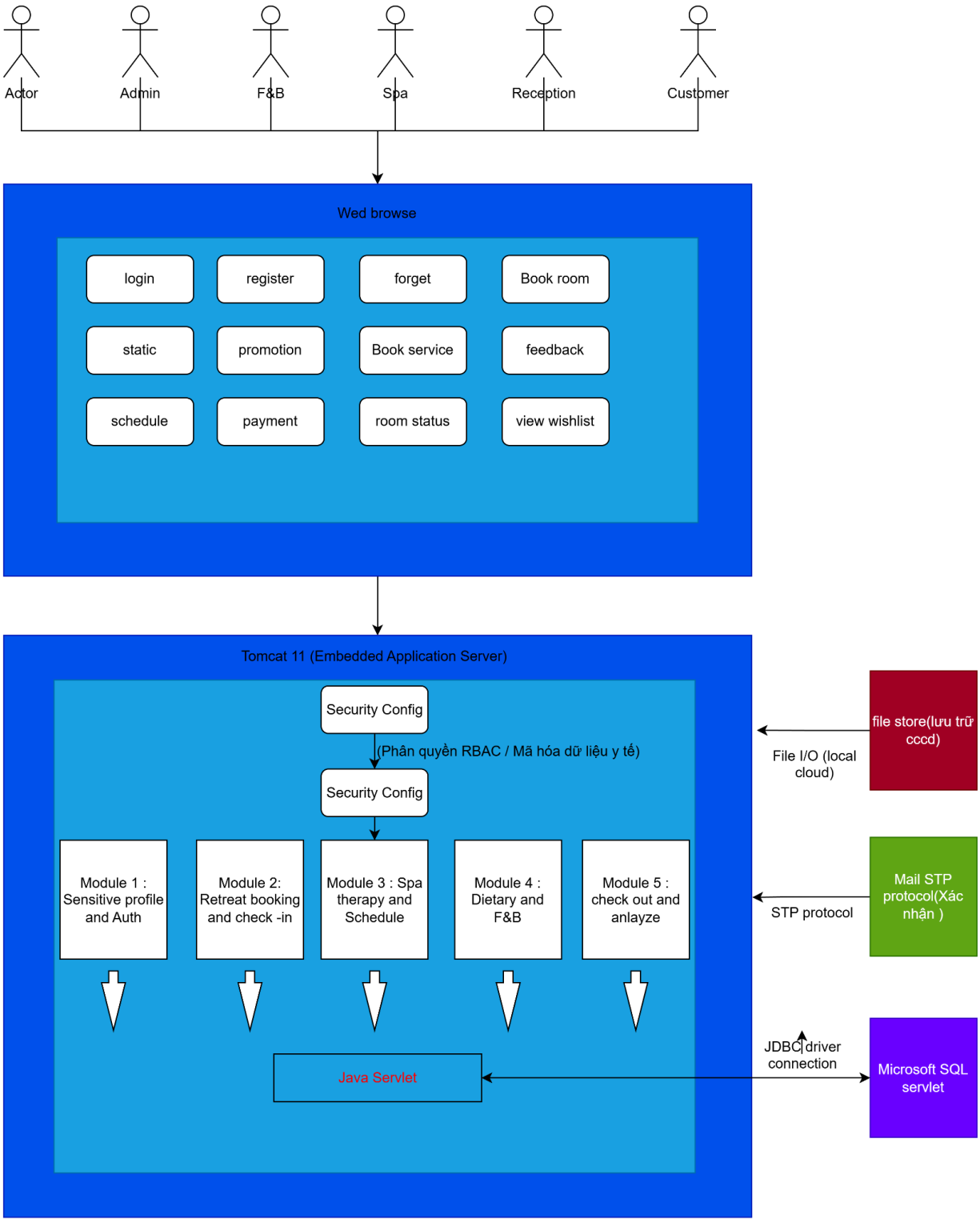
- **AHLEI Standards (The Guest Folio & Night Audit):** Used to architect the consolidated billing mechanism, where point-of-sale systems (Spa, F&B) push debts to the guest's central account.

1.3.4 Technical Standards & Documentation

- **IEEE Std 1016-2009:** IEEE Standard for Information Technology—Systems Design—Software Design Descriptions. The international standard used to structure this document.
- **RDBMS Documentation:** Technical documentation for the database engines (e.g., SQL Server) used to implement the data design and concurrent transaction locks.
- **Third-Party API Documentation:** Official documentation for Payment Gateways (Stripe/VNPay/PayPal), Calendar & Notifications (Google Calendar API/SendGrid), and Authentication (Google Identity/Facebook Login).

2. System Architecture

2.1 System Overview



React: A front-end library for building user interfaces (SPA), sending data via HTTPS to Spring Boot.

SpringBoot: A back-end framework for implementing REST API, receiving requests from the front-end, executing business logic and responding to the front-end.

File Storage: Local storage (directory `uploads/`) for avatar images and product images.

Amazon S3: A cloud storage service for uploading images.

MS SQL: A relational database management system, connected via JDBC (TCP) for data persistence and queries.

Gmail SMTP: An email service, connected via SMTP protocol (TLS 587) for sending emails (forgot password, notifications).

VNPay: A payment gateway, connected via HTTPS for processing payments and receiving webhook callbacks.

Redis cloud: A cache and session storage service, connected bidirectionally with Spring Boot for performance and session management.

Tomcat: An embedded web server / servlet container, running Spring Boot and handling HTTP requests.

2.3 Assumptions

This system is designed based on the following assumptions:

- **Frontend Technology:** React.js (v19 or later) with Tailwind CSS for building a responsive single-page application (SPA).
- **Backend Framework:** Spring Boot (v3.5 or later) with embedded Tomcat as the servlet container.
- **Operating System:**
    - **Development:** Windows 10/11
    - **Deployment:** Ubuntu 20.04 LTS (or higher) / Windows Server
- **Database Management System:** Microsoft SQL Server, connected via JDBC over TCP.
- **Java Development Kit (JDK):** Java 21 (LTS).
- **Browser Compatibility:**
    - Google Chrome (v110 or later)
    - Microsoft Edge (Chromium-based)
    - Safari (v15 or later)
- **APIs and Protocols:** RESTful APIs using JSON; HTTPS enforced for all client-server communication.
- **Authentication & Authorization:** JWT (JSON Web Tokens) integrated with Spring Security for session management and Role-Based Access Control (RBAC).
- **File Storage:** Uploaded media files (product images, avatars) are stored in a local directory (uploads/) on the application server.
- **Email Service:** Outgoing emails (password recovery, notifications) are sent via Gmail SMTP over TLS (port 587).

- **Cache & Session:** SQL server is used bidirectionally with Spring Boot for caching and distributed session management.
- **Payment Gateway:** VNPay is integrated via HTTPS for payment processing and webhook callbacks.

End-User Characteristics:

- Resort staff including receptionists, spa therapists, chefs, and resort managers/admins.
- Familiarity with standard resort management dashboards and web browsers.
- Basic computer and data management skills

- **Devices:** Desktop laptops, and tablets for floor managers; responsive UI is assumed.
- **Hosting & Infrastructure:** Deployable on cloud infrastructure or local resort servers.
- **Probable Future Changes:**
    - Direct integration with physical hardware (barcode scanners, receipt printers, POS weighing scales).
    - Integration with additional payment gateways (e-wallets, credit card terminals).
    - Mobile app extension for customer loyalty programs.
    - Migration to cloud object storage (e.g., Amazon S3) as file volume grows.
    - Advanced AI-driven inventory forecasting.

2.4 Design Constraints

The following global constraints significantly influence the architecture and implementation of the system:

- The system shall use React.js for the frontend and Java Spring Boot for the backend.
- All frontend-to-backend communication must use the Axios library over HTTPS.
- All critical POS APIs (e.g., barcode scanning, checkout) must return a response within 2–3 seconds to avoid interrupting the customer checkout flow.
- The system must use JWT combined with Spring Security for authentication, session management, and Role-Based Access Control (RBAC).
- The system must enforce HTTPS for all external communication to ensure data encryption in transit.

- The backend must comply with RESTful API design principles and expose versioned endpoints (e.g., /api/v1/...).
- The system must use Microsoft SQL Server as the primary relational database, connected via JDBC.
- Uploaded media files (product images, category icons, avatars) must be stored in the local uploads/ directory on the application server.
- All outgoing automated emails (e.g., password recovery, notifications) must be sent via Gmail SMTP over TLS (port 587).
- Redis Cloud must be integrated with Spring Boot for caching frequently accessed data and managing user sessions.
- All user input must be validated on both the client side (React) and server side (Spring Boot) to prevent malformed or malicious data.
- The application must be compatible with modern web browsers including Chrome, Firefox, Edge, and Safari.
- The application should be structured to support containerized deployment (e.g., Docker) for easier scaling in the future.

3. Software Architecture Design

3.1 Architectural Representation

Ngu Son Retreat Management System (NSRMS) architecture is represented using the 4+1 Architectural View Model (Kruchten, 1995), as referenced in Gomaa's Software Modeling and Design. The following views are used:

View	Purpose	UML Diagrams Used
Logical View	Internal structure: packages, classes, responsibilities, and interactions between software components	Class Diagram
Process View	Runtime behavior: thread management, concurrency, synchronization, and communication between internal components	Activity Diagram
Development View	Organization of the source code: modules, layers, and package dependencies	Package Diagram
Deployment View	Physical mapping of software components to hardware nodes (covered in Section 2.2)	Deployment Diagram

The system follows a Layered Architecture Pattern with strict dependency rules.

3.2 Software Architecture

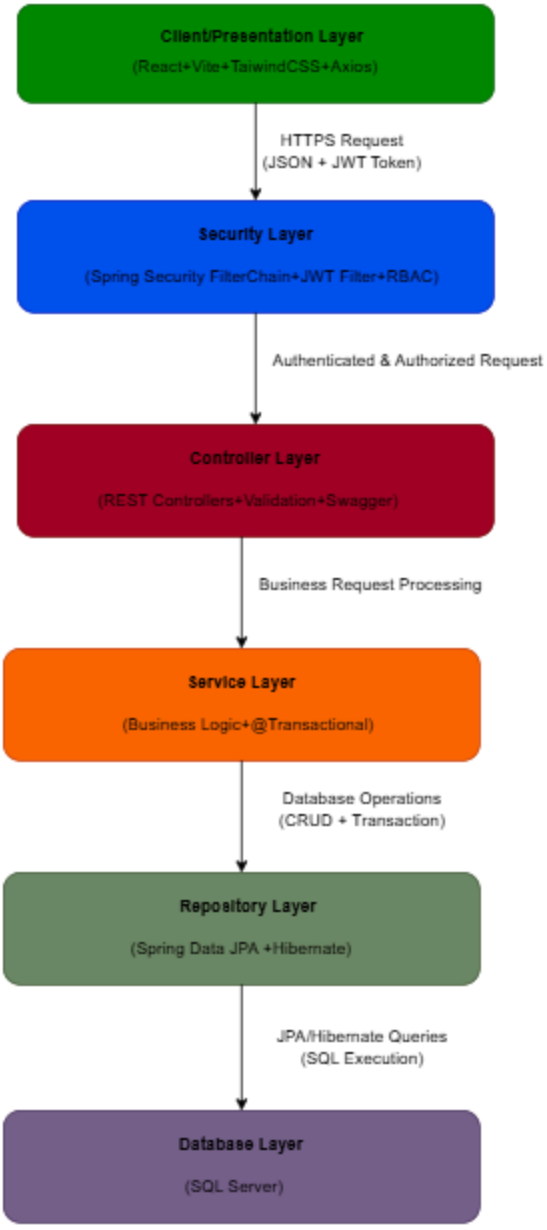

Client/Presentation Layer
(React+Vite+TaiwindCSS+Axios)
HTTPS Request
(JSON + JWT Token)
Security Layer
(Spring Security FilterChain+JWT Filter+RBAC)
Authenticated & Authorized Request
Controller Layer
(REST Controllers+Validation+Swagger)
Business Request Processing
Service Layer
(Business Logic+@Transactional)
Database Operations
(CRUD + Transaction)
Repository Layer
(Spring Data JPA +Hibernate)
JPA/Hibernate Queries
(SQL Execution)
Database Layer
(SQL Server)

Figure 3-1: NSRMS Layered Architecture Pattern

Layer	Responsibility
Client/Presentation	Renders UI, handles user interaction, sends HTTP requests via Axios with JWT interceptor.
Security	Intercepts every request, validates JWT token, sets SecurityContext, enforces role-based access using Spring Security Filter Chain.
Controller	Handles incoming HTTP requests and returns responses. Uses annotations and maps API endpoints.
Service	Contains the core business logic of the application. Processes data, validates input and coordinates between Controller and Repository.
Repository	Handles communication with the database using Spring Data JPA.
Database	Stores persistent application data in SQL Server relational database.

3.2.1 Process View

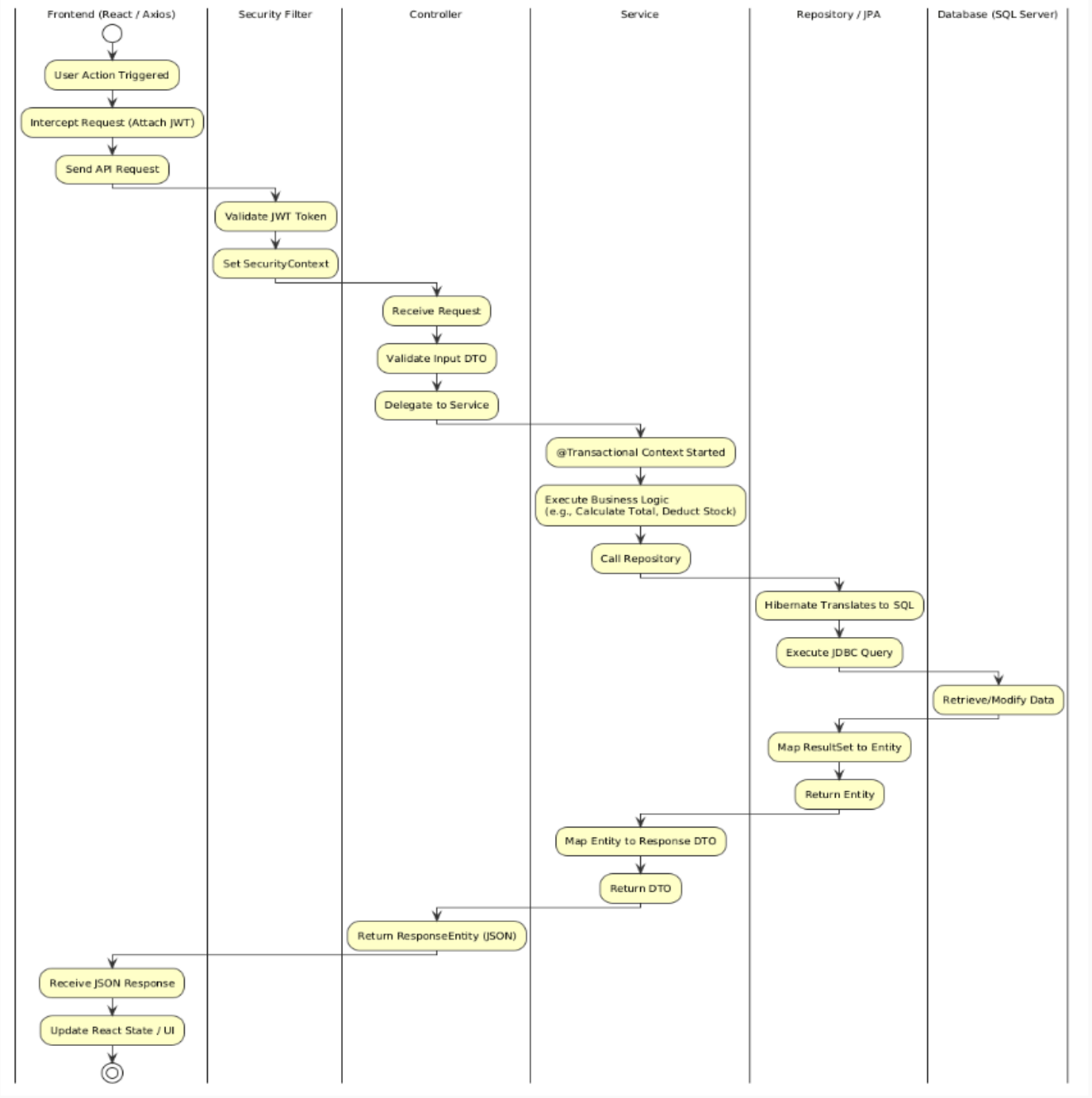


Figure 3-2: NSRMS Process Overview

3.2.2 Logical View

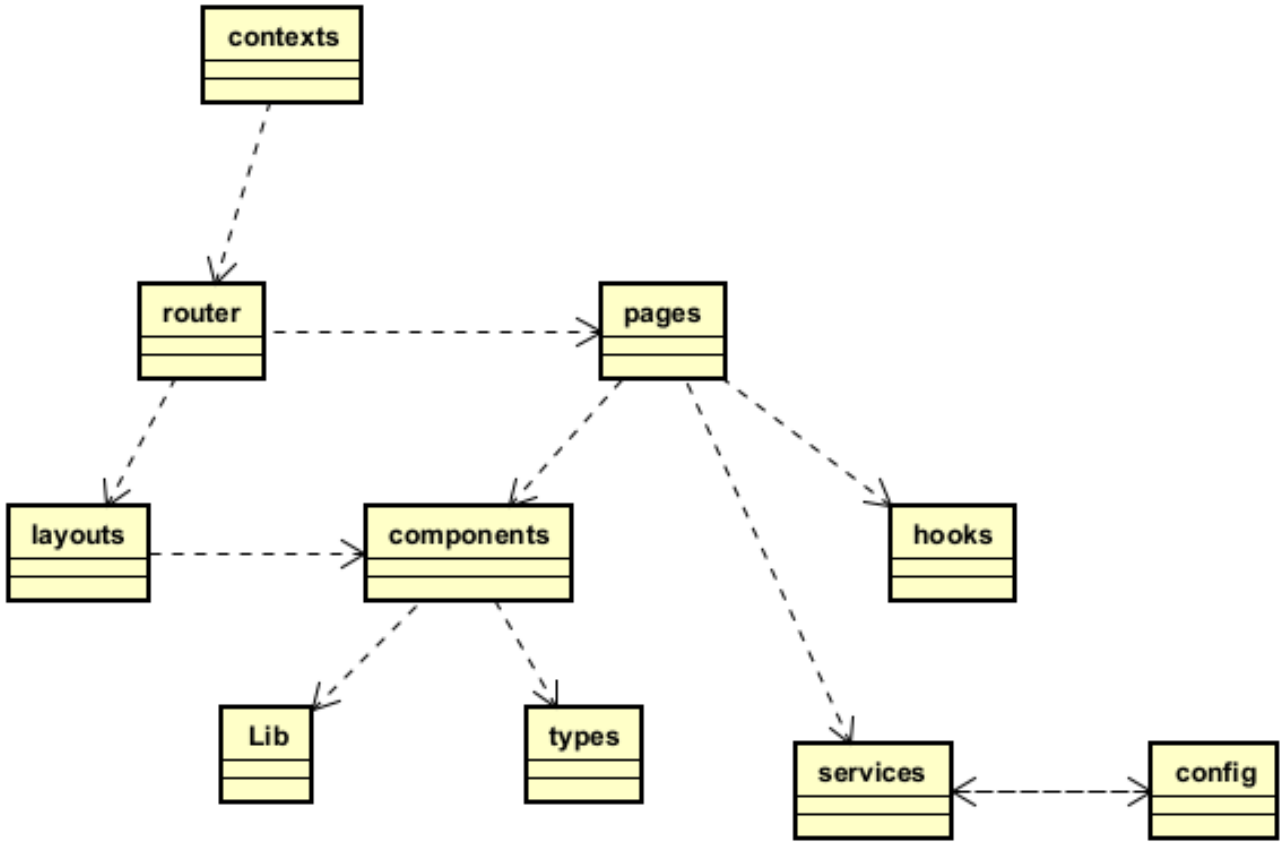


Figure 3-3: NSRMS FrontEnd Logical Overview

#	**Responsibility**
Contexts	React Context providers for authentication, user session, and global state management.
Config	Application configuration: environment variables, API endpoints, constants.
Router	React Router DOM routing, ProtectedRoute, RoleRoute (RBAC on frontend)
Layouts	Page layout wrappers: sidebar, header, navigation structure
Pages	Feature-oriented pages implementing business workflows and user interactions.
Components	Reusable UI components, shared presentation elements, and theme management.
Services	API communication layer using Axios HTTP client with JWT interceptor and domain-based service modules.

Hooks	Custom React hooks for reusable state management, asynchronous operations, and business features.
Lib	Shared utility functions and helper libraries.
Types	Shared type definitions, constants, and application models.

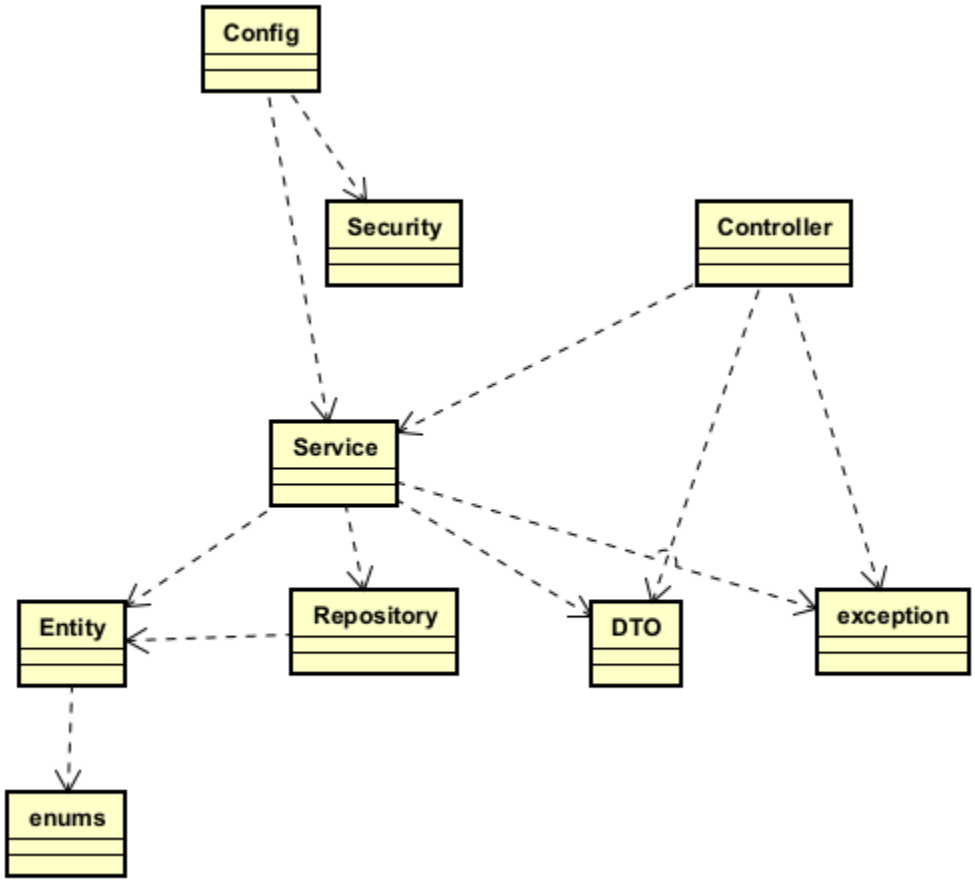


Figure 3-4: NSRMS BackEnd Logical Overview

#	**Responsibility**
Config	Application-level configurations for security, API documentation, persistence, and external integrations.
Security	JWT filter chain, token provider, password encoder, stateless session

Controller	REST controllers handling HTTP requests, input validation, and API responses.
Service	Core business logic, transaction management, and coordination between application layers.
Repository	Spring Data JPA interfaces for database CRUD and custom JPQL queries
Entity	JPA entity models mapped to relational database tables via Hibernate ORM.
DTO	Request/Response data transfer objects for API communication
Enum	Enumerated value types representing application states and business constants.
Exception	Custom exception handling for validation errors, business rules, and resource management.

3.2.3 Development View

3.2.3.1 Package Diagram

3.2.3.1.1 Back-end package diagram

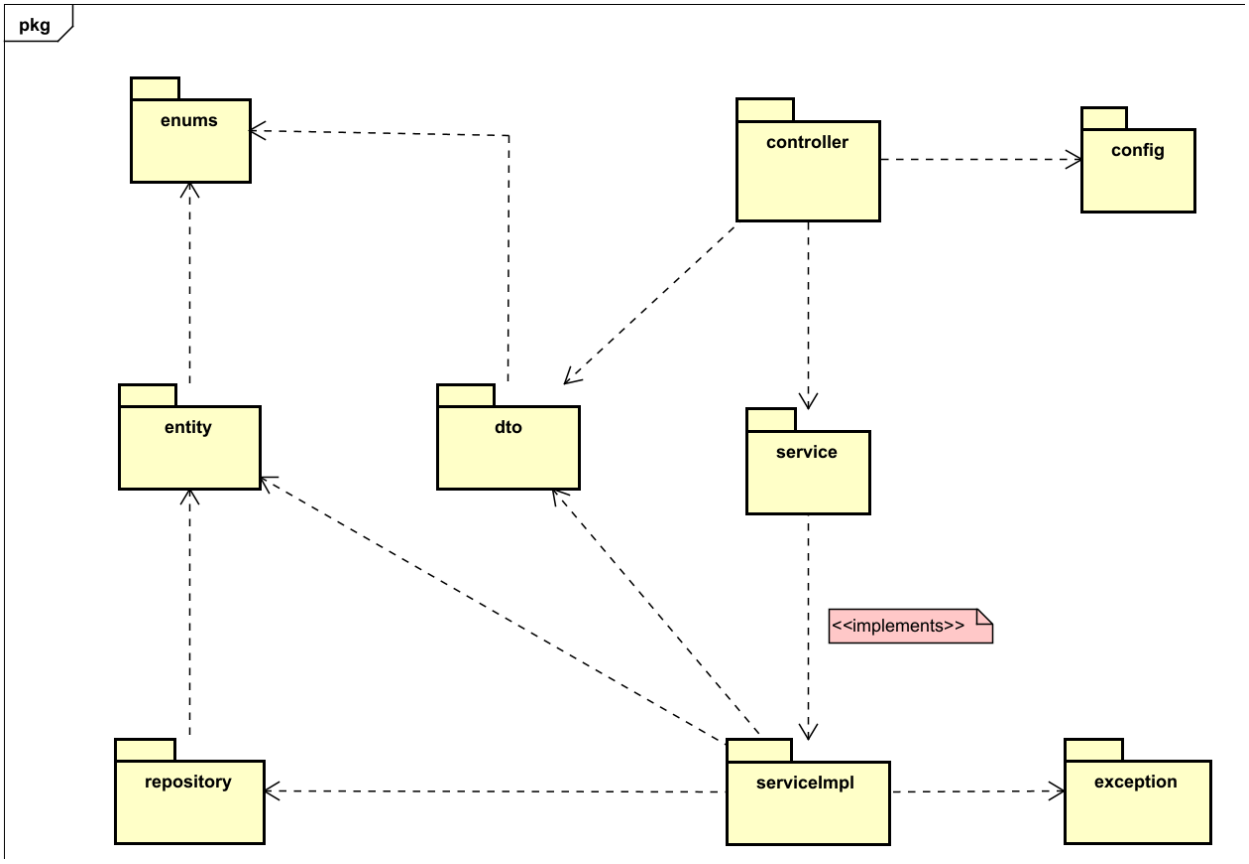


Figure 3-1: NSRMS Back-End Package Diagram

Packages Description

No	Package	Description
01	Config	Còniguration: Security, JWT, CORS, Web, OpenAPI, JPA Auditing
02	Controller	REST Controllers (API endpoints)
03	DTO	Data Transfer Objects (request/response)
04	Entity	JPA Entities
05	Enums	Enum types
06	Exception	Custom exceptions and global exception handler
07	Repository	Spring Data JPA repositories
08	Service	Service interfaces
09	ServiceImpl	Service implementations

3.2.3.1.2 Front-end package diagram

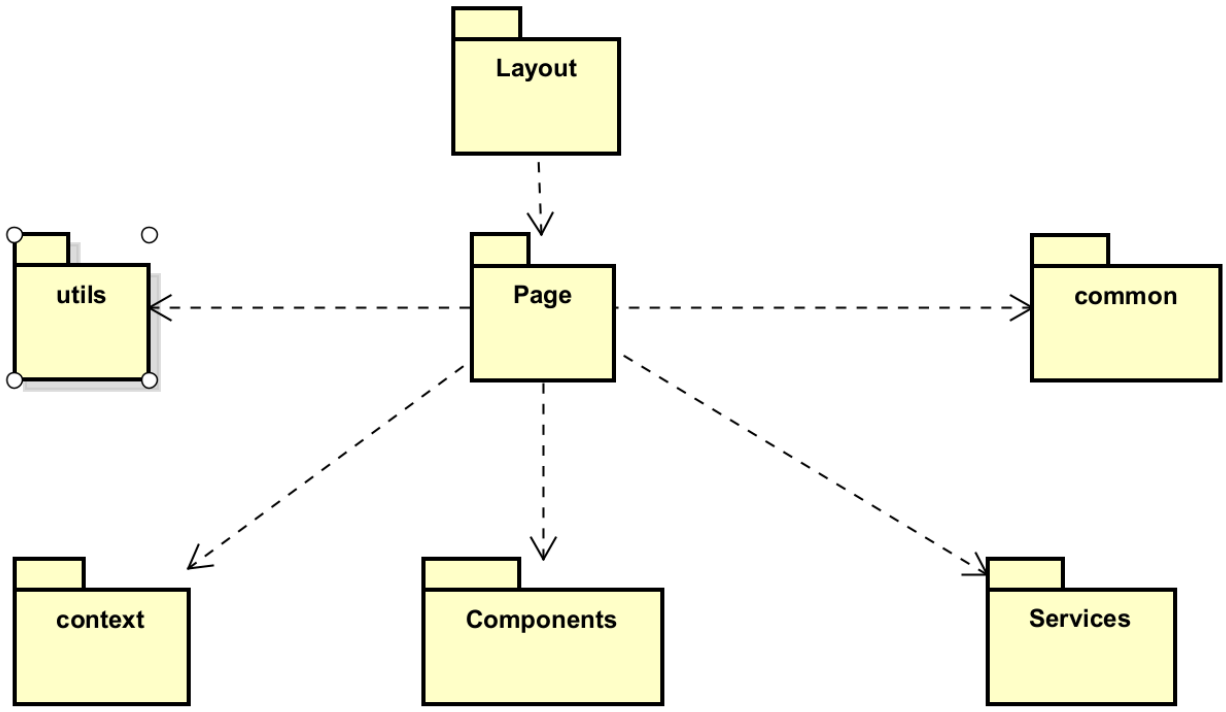


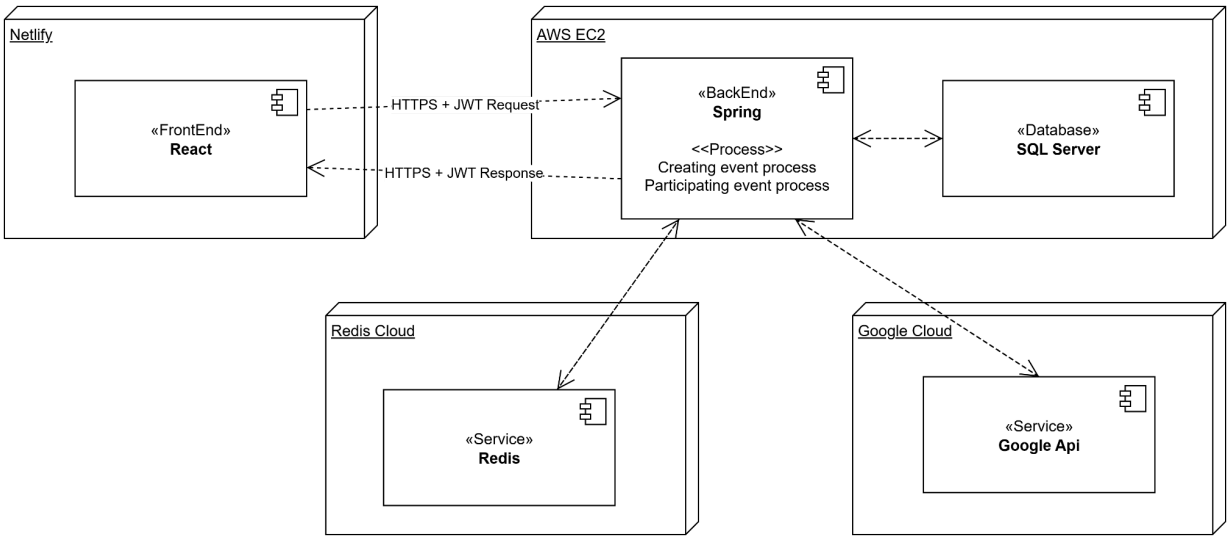
Figure 3-2: NSRMS Front-End Package Diagram

Packages Description

No	Package	Description
01	service	Handles business logic, abstracts API interactions, and manages all endpoint configurations for components and pages

02	common	Contains reusable utilities, constants, helpers, and shared logic used across the application
03	component	Contains reusable UI components used across pages (e.g., buttons, inputs, modals)
04	context	Manages global application state using React Context API
05	layout	Defines the main layout components such as headers, footers, and sidebars
06	page	Contains full page-level components mapped to routes

3.2.4 Deployment View



Configuration Description

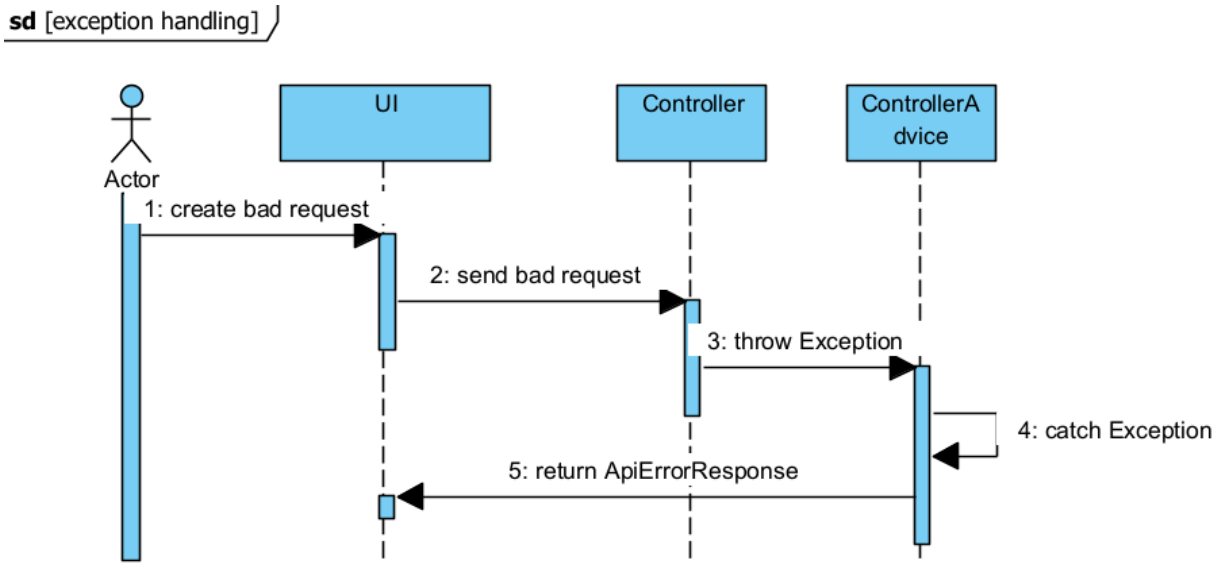
No	Component/Node	Description
01	Netlify	Hosting platform for deploying the React-based Frontend. It serves the static assets (HTML, JS, CSS) and handles HTTPS communication to the backend.
02	React (FrontEnd)	A Single Page Application (SPA) developed in React, responsible for interacting with users and sending requests to the backend via HTTPS.
03	AWS EC2	Cloud server hosting the backend and database. Acts as the main execution environment for application logic and data management.

04	Spring (BackEnd)	Java Spring Boot application that handles business logic, exposes REST APIs, and processes frontend requests. Deployed on the AWS EC2 node.
05	SQL Server (Database)	Relational database system that stores persistent data (e.g., users, events, tickets). Connected to the backend via internal service calls on the same EC2 instance.
06	Redis Cloud	Cloud-based Redis service used to store temporary data (sessions, tokens, cache) to enhance system performance.
07	Google Cloud	Utilized for services such as Gmail (email sending), Google Authentication (user login), and Google Drive (file storage and sharing).

4. Detailed Component Design

4.1 Common Design

4.1.1 Exception handling

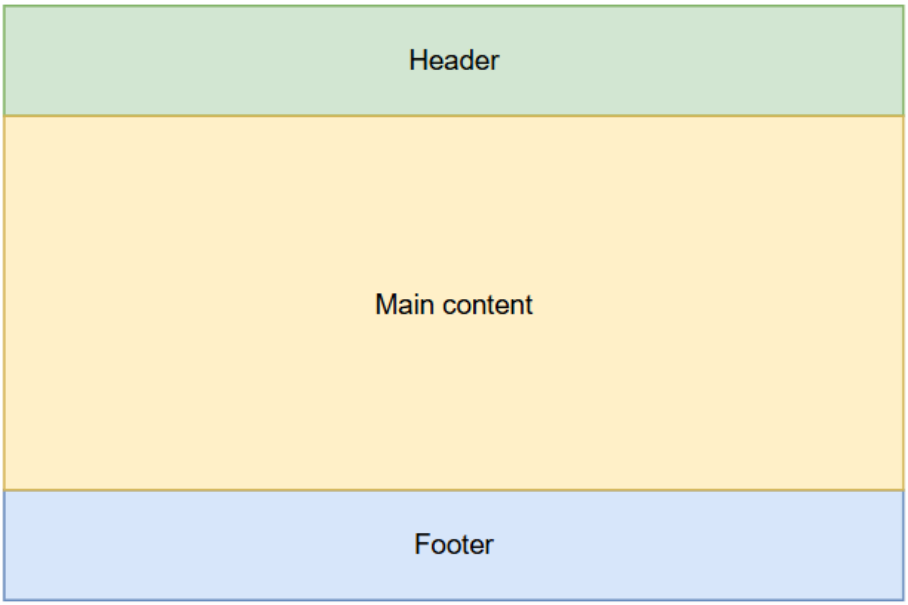


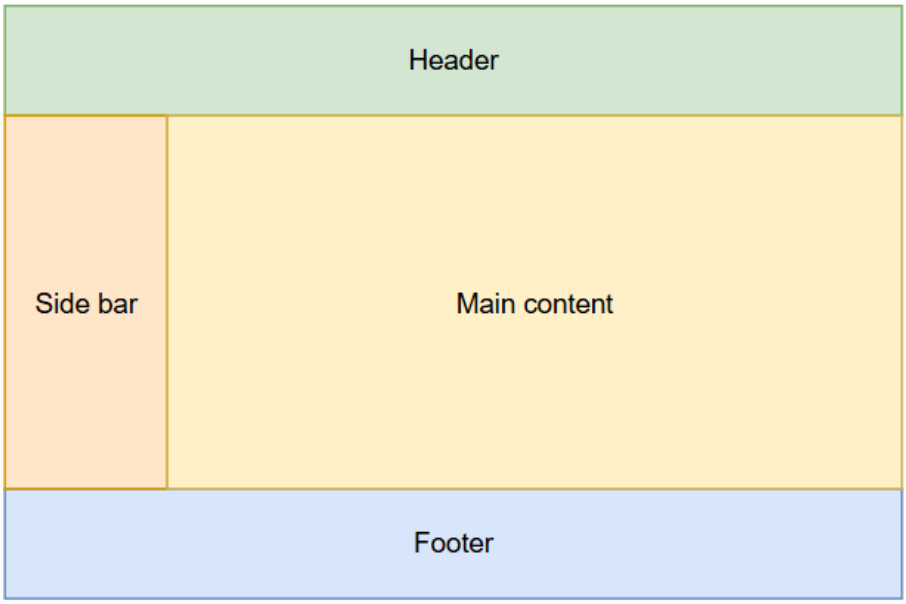
The system will use a centralized exception handling mechanism using @ControllerAdvice in Spring Boot.

- All exceptions thrown from controllers or services will be caught and converted into standard ApiErrorResponse objects with fields:
    - timestamp, status, errorCode, message, traceId (if needed).
- Custom exceptions include:

    - ResourceNotFoundException
    - BadRequestException
    - UnauthorizedException
    - BusinessException
- Example handling flow:
    - Service -> throw new BusinessException("Email already in use")
    - ControllerAdvice -> catches -> builds HTTP 400 with message.

4.1.2 UI Structures





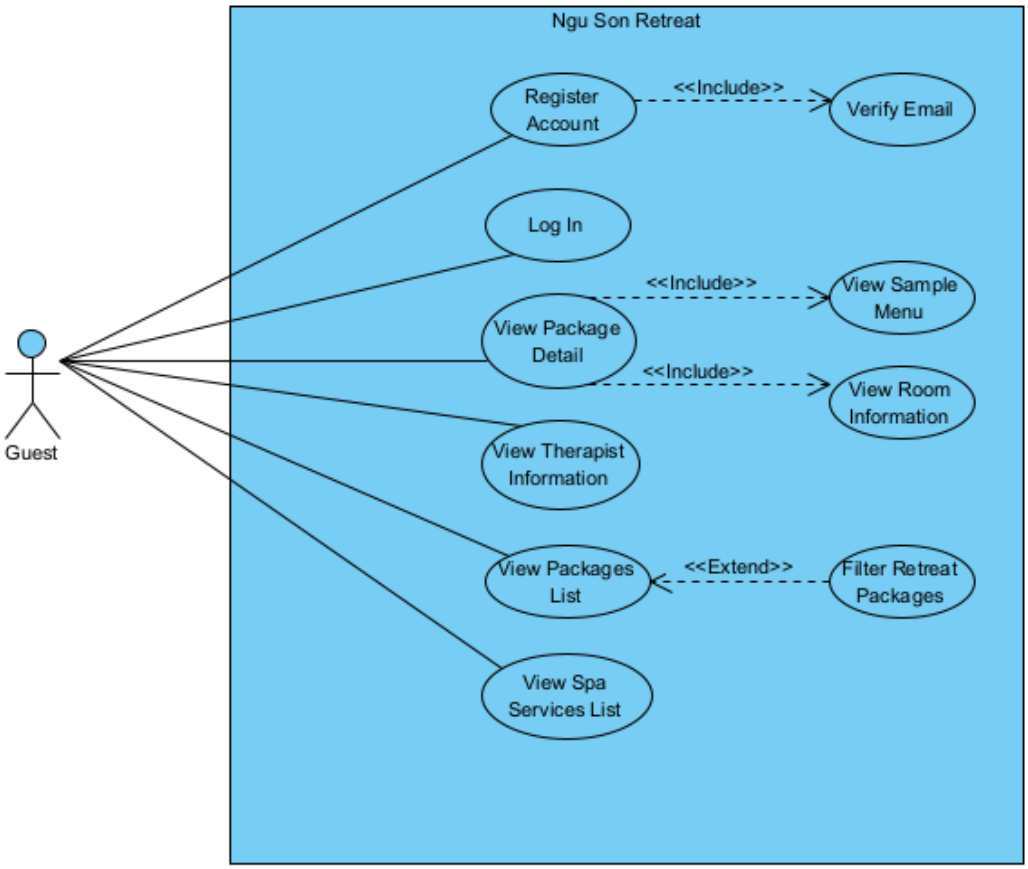
- The UI is implemented using ReactJS and follows a component-based architecture.
- Common layout structure includes:
    - Header: logo, user menu
    - Sidebar (for Admin/User views)
    - Main content: renders based on route
    - Footer
- Shared UI components:
    - Button, Modal, FormInput, Notification, Spinner, Dialog
- Page routing is managed using React Router.
- All API errors will be handled globally with toast messages or error components.

4.1.3 Transaction Management

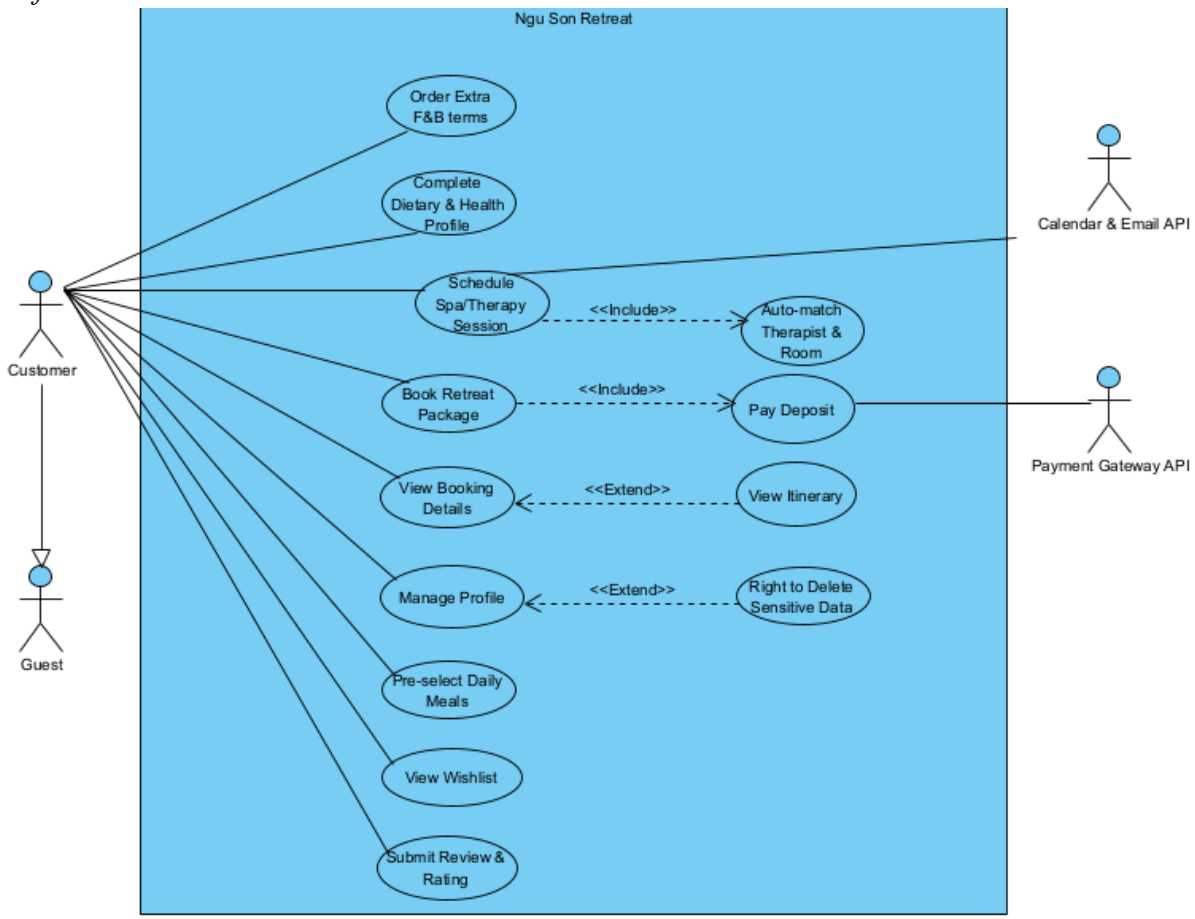
- The backend uses Spring's declarative transaction management via @Transactional.
- All service-level operations that modify database state are wrapped in a transaction.
- In case of exception, transactions are automatically rolled back.
- Read-only operations are optionally marked as @Transactional(readOnly = true) to improve performance.

4.2 User & Authentication

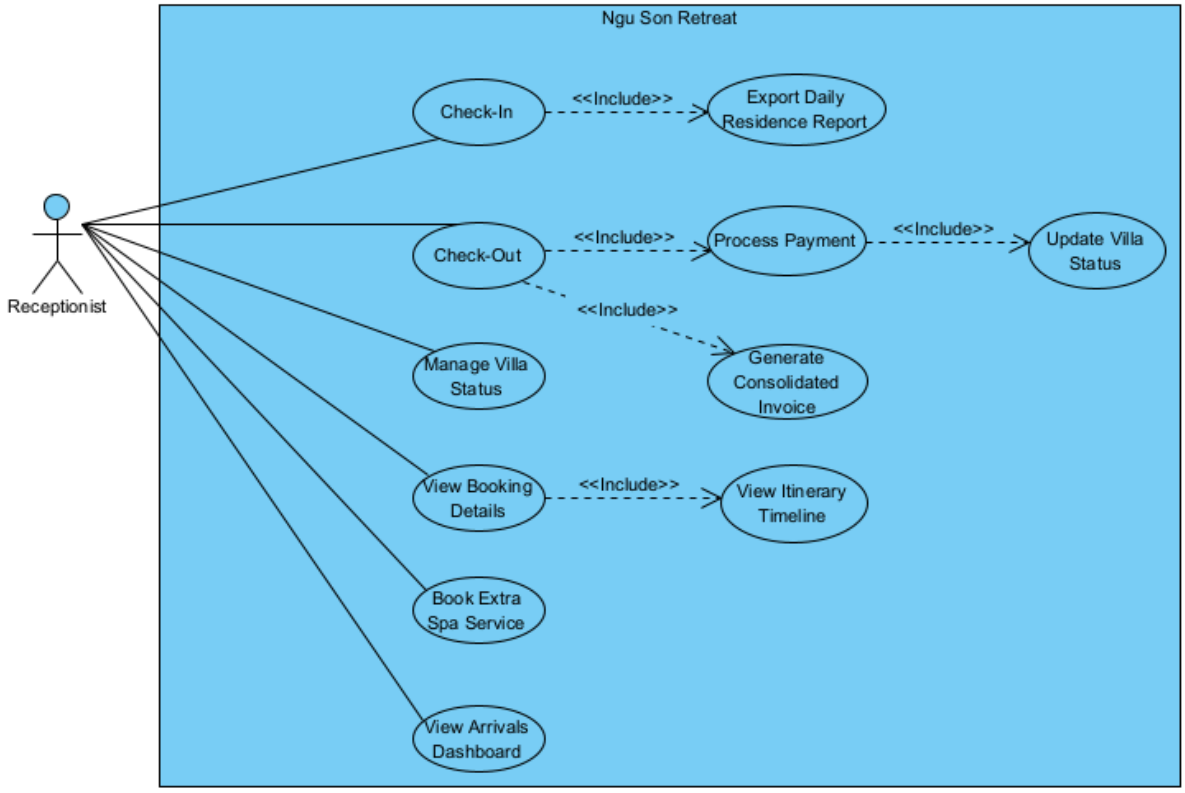
UCs for Guest


Ngu Son Retreat
Register Account
<<Include>>
Verify Email
Log In
<<Include>>
View Sample Menu
View Package Detail
<<Include>>
View Room Information
View Therapist Information
View Packages List
<<Extend>>
Filter Retreat Packages
View Spa Services List
Guest

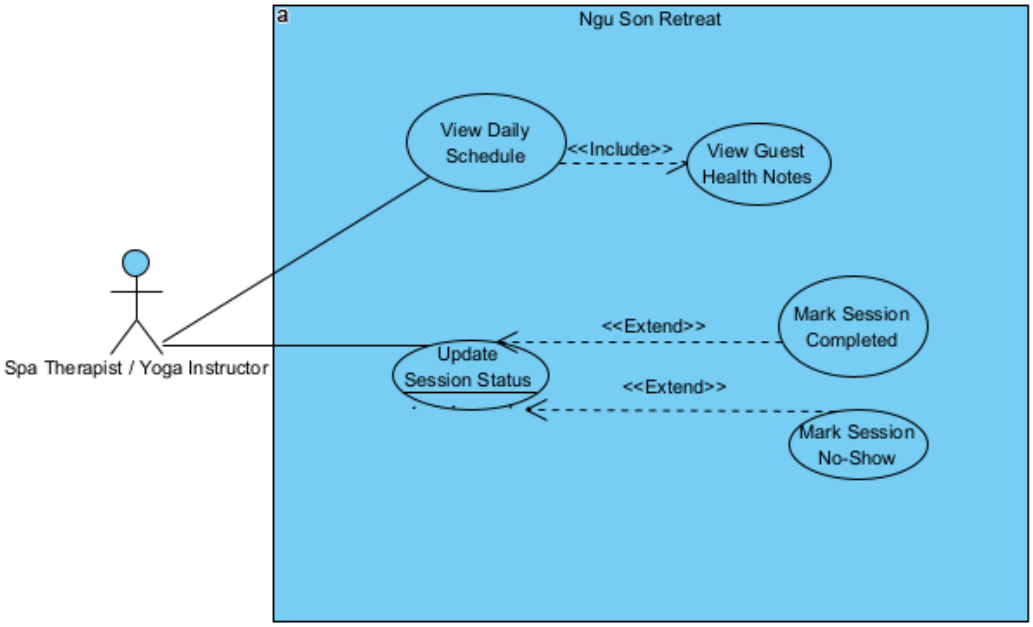
UCs for Customer



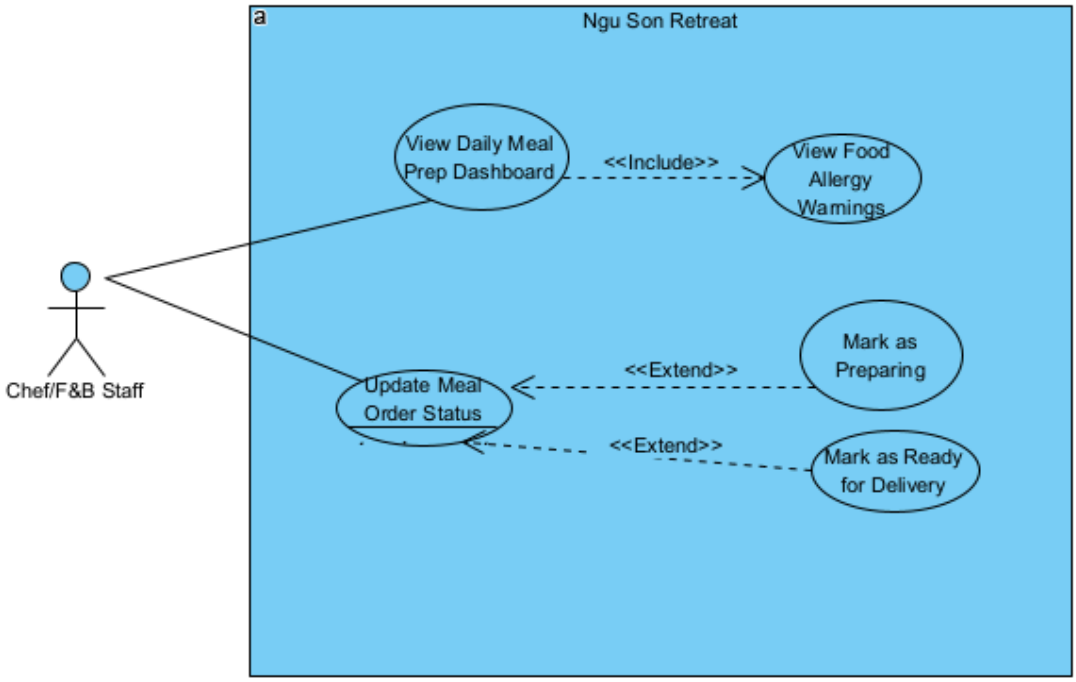
UCs for Receptionist



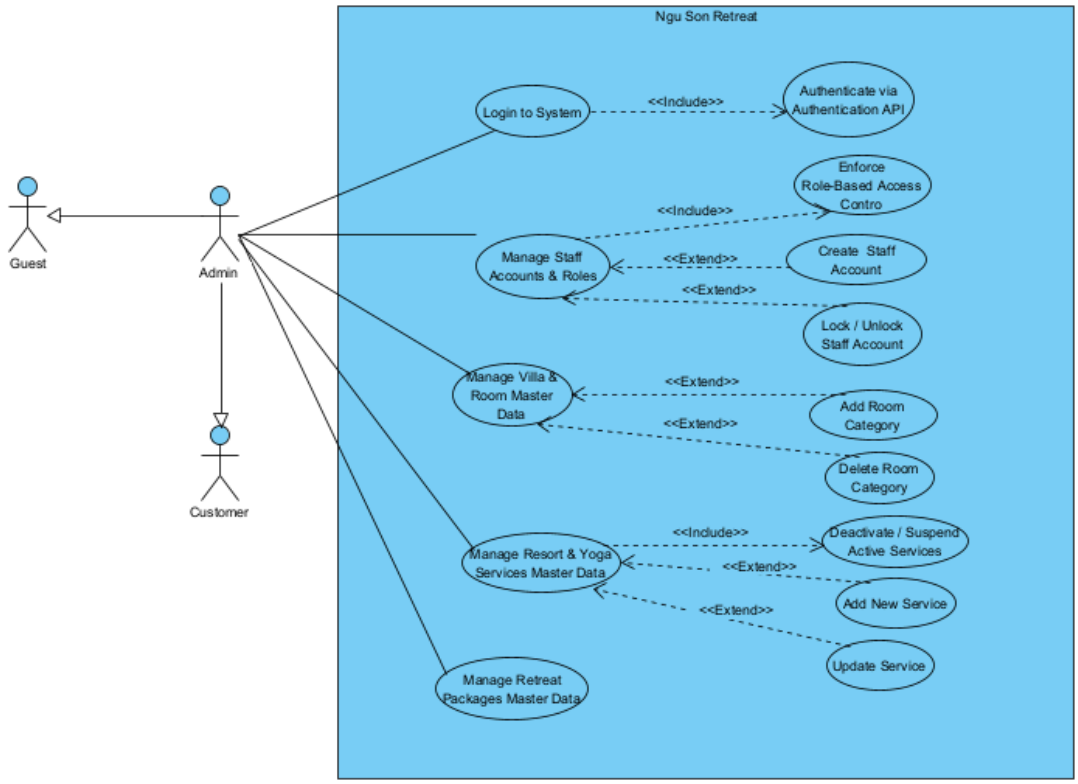
UCs for Spa Therapist / Yoga Instructor



UCs for Chef


a
Ngu Son Retreat
View Daily Meal Prep Dashboard
<<Include>>
View Food Allergy Warnings
Chef/F&B Staff
Update Meal Order Status
<<Extend>>
Mark as Preparing
<<Extend>>
Mark as Ready for Delivery

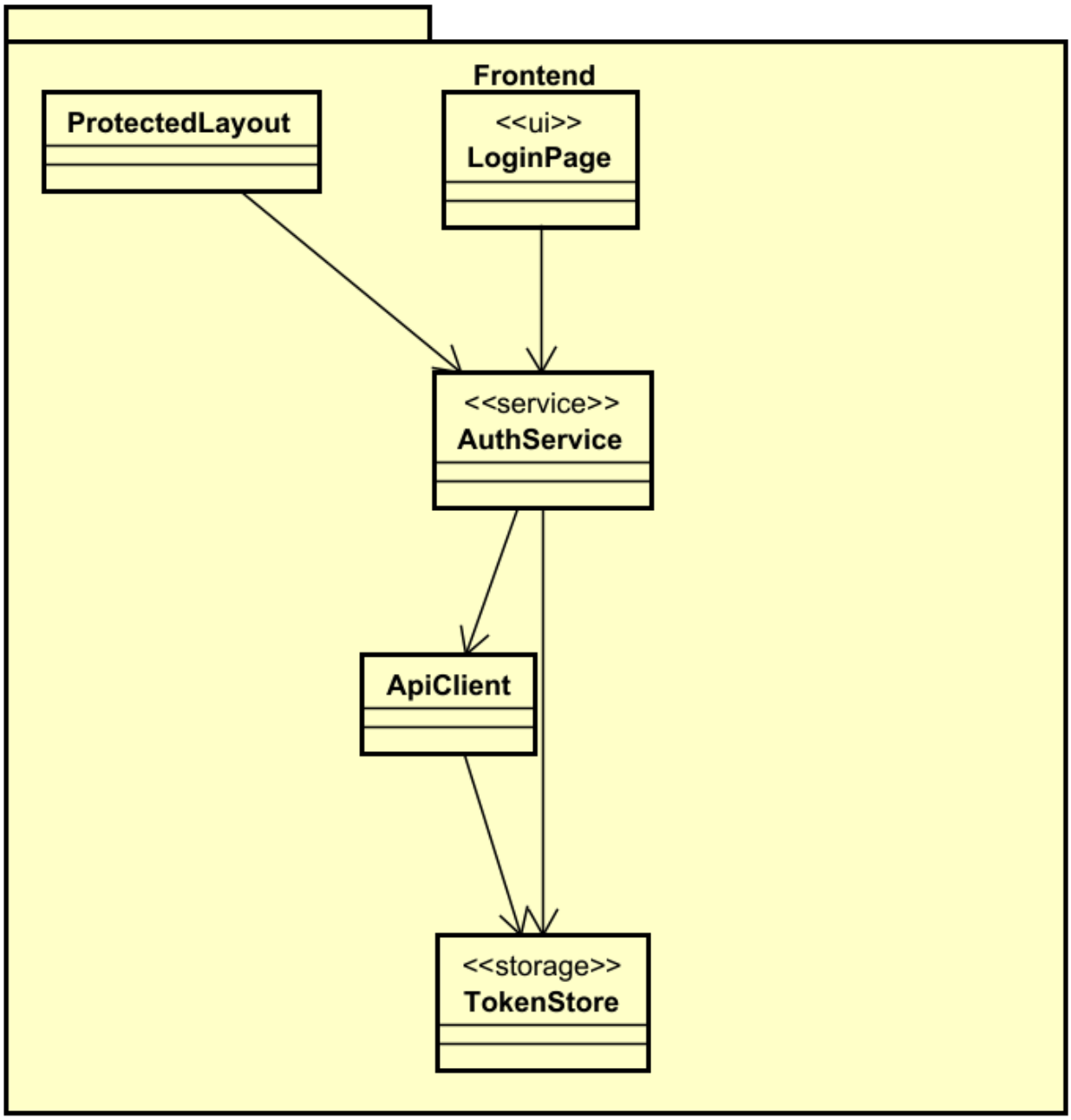
UCs for Admin



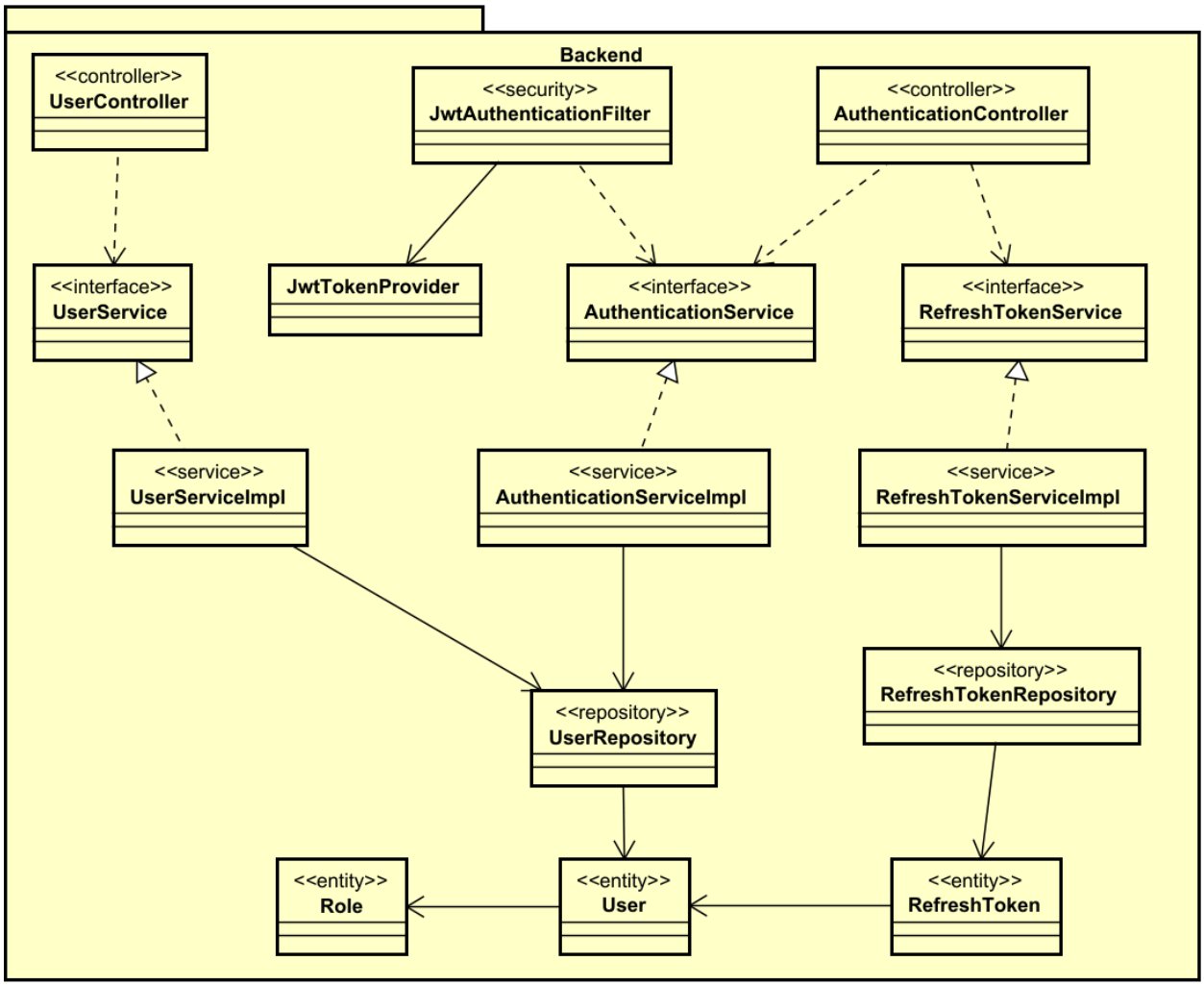
UCs for Business Manager

4.2.1 Class Design

4.2.1.1 Classes Structure



Front-end Class Diagram



Back-end Class Diagram

4.2.1.2 Classes Description

4.2.1.2.1 UserController Class

Class	**UserController**		
Description	Handles user-related API requests (profile, avatar, change password). Requires authentication.		
Base Class	None		
Constructor	UserController(UserService userService)		
Prototype	@RestController		
Source File	fu.se.smms.controller.UserController.java		
Namespace	fu.se.smms.controller		
Attributes	**Name**	**Type**	**Description**

	userService	User Service	Business logic for user operations	
Methods	**Name**	**Input**	**Output**	**Description**
	getProfile	Authentication	ResponseEntity <UserProfileDTO>	Returns profile for current user
	forgotPassword	Authentication,UpdateProfileRequestDTO	ResponseEntity <UserProfileDTO>	Updates profile
	resetPassword	Authentication,MultipartFile	ResponseEntity <UserProfileDTO>	Uploads avatar
	getProfile	Authentication, ChangePasswordRequestDTO	ResponseEntity <String>	Changes password

4.2.1.2.2 AuthenticationController Class

Class	**AuthenticationController**			
Description	Handles login and token refresh requests. Authenticates credentials, issues JWT access token, manages refresh token via httpOnly cookie.			
Base Class	None			
Constructor	AuthenticationController(AuthenticationService, JwtTokenProvider, RefreshTokenService, UserDetailsService)			
Prototype	@RestController			
Source File	fu.se.smms.controller.AuthenticationController.java			
Namespace	fu.se.smms.controller			
Attributes	**Name**	**Type**	**Description**	
	authenticationService	AuthenticationService	Validates credentials	
	jwtTokenProvider	JwtTokenProvider	Generates access tokens	
	refreshTokenService	RefreshTokenService	Manages refresh tokens	
	userDetailsService	UserDetailsService	Loads user for token generation	
Methods	**Name**	**Input**	**Output**	**Description**

	login	LoginRequestDTO, Response	ResponseEntity <LoginResponse>	Authenticates & sets refresh cookie
	refresh	Request, Response	ResponseEntity <LoginResponse>	Issues new access token from cookie
	logout	Request, Response	ResponseEntity <Void>	Revokes token & clears cookie
	me	UserDetailDTO	ResponseEntity <UserDTO>	Returns current user

4.2.1.2.3 UserService class

Class	**UserService**			
Description	Contains business logic for user management. Provides profile operations and staff CRUD.			
Base Class	None(Interface)			
Constructor	N/A			
Prototype	@Service			
Source File	fu/se/smms/service/UserService.java			
Namespace	fu.se.smms.service			
Attributes	**Name**	**Type**	**Description**	
Methods	**Name**	**Input**	**Output**	**Description**
	getProfile	String username	UserProfileDTO	Returns user profile
	updateProfile	String username, UpdateProfileReq	UserProfileDTO	Updates profile
	uploadAvatar	String username, MultipartFile	UserProfileDTO	Uploads avatar
	changePassword	String username, ChangePassReq	void	Changes password
	getAllStaff	None	List<StaffDTO>	Maps all users with role to StaffDTO
	createStaff	StaffDTO	StaffDTO	Creates User from request, encodes password
	updateStaff	String prefixId, StaffDTO	StaffDTO	Updates staff by EMP prefix ID
	deleteStaff	String prefixId	void	Soft delete (status=false)

4.2.1.2.4 AuthenticationService class

Class	**AuthenticationService**			
Description	Manages the logic for authenticating users. Validates credentials via AuthenticationManager, loads User entity and returns UserDetailDTO.			
Base Class	None(Interface)			
Constructor	N/A			
Prototype	@Service			
Source File	fu/se/smms/service/AuthenticationService.java			
Namespace	fu.se.smms.service			
Attributes	**Name**	**Type**	**Description**	
Methods	**Name**	**Input**	**Output**	**Description**
	authenticate	LoginRequestDTO	UserDetailDTO	Validates credentials, returns principal
	getCurrentUser	String username	UserDTO	Returns current user DTO
	getUserEntity	String username	User	Returns User entity

4.2.1.2.5 RefreshTokenService class

Class	**RefreshTokenService**			
Description	Manages the logic for creating and validating refresh tokens. Issues UUID-based tokens, revokes on logout.			
Base Class	None (interface)			
Constructor	N/A			
Prototype	@Service			
Source File	fu/se/smms/service/impl/RefreshTokenService .java			
Namespace	fu.se.smms.service			
Attributes	**Name**	**Type**	**Description**	
Methods	**Name**	**Input**	**Output**	**Description**
	createRefreshToken	User	RefreshToken	Creates and saves new refresh token

	findByToken	String token	RefreshToken	Finds valid, non-revoked token
	revokeByToken	String token	void	Revokes token
	revokeAllByUser	User	void	Revokes all tokens for user

4.2.1.2.6 **UserRepository**

Class	**UserRepository**			
Description	Data access layer for User entities. Extends JpaRepository.			
Base Class	JpaRepository<User, Integer>			
Constructor	N/A			
Prototype	@Repository			
Source File	fu/se/smms/repository/UserRepository.java			
Namespace	fu.se.smms.repository			
Attributes	**Name**	**Type**	**Description**	
Methods	**Name**	**Input**	**Output**	**Description**
	findByEmail	String email	Optional<User>	Finds user by email
	findByResetPasswordToken	String token	Optional<User>	Finds user by reset token
	findByUsername	String username	Optional<User>	Finds user by username

4.2.1.2.7 RefreshTokenRepository

4.2.1.2.9 Role Entity

Class	**Role**			
Description	Represents user permissions/roles (ADMIN, CUSTOMER,...). Each User has one Role.			
Base Class	BaseAuditableEntity			
Constructor	NoArgs / AllArgs / Builder (Lombok)			
Prototype	@Entity			
Source File	fu.se.smms.entity.Role.java			
Namespace	fu.se.smms.entity			
Attributes	**Name**	**Type**	**Description**	
	roleId	Integer	Primary key, auto-generated	
	roleName	String	Unique role name	
	users	List<User>	Users assigned to this role	
Methods	**Name**	**Input**	**Output**	**Description**

4.2.1.2.10 RefreshToken Entity

Class	**RefreshToken**			
Description	Stores refresh tokens used to re-authenticate users without requiring login credentials. Linked to User entity.			
Base Class	BaseAuditableEntity			
Constructor	NoArgs / AllArgs / Builder (Lombok)			
Prototype	@Entity			
Source File	fu.se.smms.entity.RefreshToken.java			
Namespace	fu.se.smms.entity			
Attributes	**Name**	**Type**	**Description**	
	tokenId	Integer	Primary key, auto-generated	
	user	User	User owning this reset token	
	token	String	Unique reset token string	
	expiresAt	LocalDateTime	Expiration timestamp	
	used	Boolean	Whether token has been consumed	
	createdAt	LocalDateTime	Token creation timestamp	
Methods	**Name**	**Input**	**Output**	**Description**
	onCreate	None	void	Sets createdAt and updatedAt before insert
	isExpired	None	boolean	Returns true if expired or revoked

4.2.1.2.11 JwtAuthenticationFilter

Class	**JwtAuthenticationFilter**			
Description	Intercepts requests to validate JWTs. Extracts Bearer token, validates via JwtTokenProvider, loads UserDetails, sets SecurityContext. Depends on JwtTokenProvider and UserDetailsService (AuthenticationService in diagram context).			
Base Class	extends OncePerRequestFilter			
Constructor	JwtAuthenticationFilter(HandlerExceptionResolver, JwtTokenProvider, UserDetailsService)			
Prototype	@Component			
Source File	fu.se.smms.config.JwtAuthenticationFilter.java			
Namespace	fu.se.smms.config			
Attributes	**Name**	**Type**	**Description**	
	jwtTokenProvider	JwtTokenProvider	Validates and parses JWT	
	userDetailsService	UserDetailsService	Loads user for SecurityContext	
Methods	**Name**	**Input**	**Output**	**Description**
	doFilterInternal	request,response,filterchain	void	Validates Bearer token, sets authentication

4.2.1.2.12 JwtTokenProvider

Class	**JwtTokenProvider**
Description	Responsible for generating and validating JWT strings. Uses HMAC-SHA with secret from config.
Base Class	None
Constructor	Default (Spring-managed)
Prototype	@Service
Source File	fu.se.smms.config.JwtTokenProvider.java

Namespace	fu.se.smms.config			
Attributes	**Name**	**Type**	**Description**	
	jwtSecret	String	Base64-encoded HMAC key	
	accessExpirationMs	long	Token validity in ms	
Methods	**Name**	**Input**	**Output**	**Description**
	generateToken	UserDetailDTO	String	Creates signed JWT with subject, role, expiry
	getUsernameFromToken	String	String	Extracts subject from token
	validateToken	String	boolean	Verifies signature and expiry

4.2.1.2.13 LoginPage

Class	**LoginPage**			
Description	The visual interface where users enter credentials. Calls AuthService to login, stores token, redirects based on role.			
Base Class	None			
Constructor	N/A			
Prototype	React component			
Source File	page/Login/Login.html			
Namespace	@/page/Login			
Attributes	**Name**	**Type**	**Description**	
	username	String	Username input state	
	password	String	Password input state	
	loading	String	Submit loading state	
	error	String	Error message state	
Methods	**Name**	**Input**	**Output**	**Description**

	handleLogin	Event	void	Calls authService, stores token, fetches profile, navigates

4.2.1.2.14 ProtectedLayout

Class	**ProtectedLayout**			
Description	Structural component that wraps routes requiring authentication. Redirects to / if not authenticated.			
Base Class	None			
Constructor	N/A			
Prototype	React component			
Source File	components/ProtectedLayout.js			
Namespace	@/components			
Attributes	**Name**	**Type**	**Description**	
Methods	**Name**	**Input**	**Output**	**Description**

4.2.1.2.15 AuthService

Class	**AuthService**		
Description	Central service for handling login/logout logic and managing authentication state. Coordinates between ApiClient, TokenStore, and auth endpoints.		
Base Class	None		
Constructor	N/A		
Prototype	Service module		
Source File	services/authService.js		
Namespace	@/services		
Attributes	**Name**	**Type**	**Description**

Methods	Name	Input	Output	Description
	login	username, password	Promise	POST /auth/login, stores token
	register	data	Promise	POST /auth/register
	refresh	None	Promise<String>	POST /auth/refresh, returns new token
	logout	None	Promise	POST /auth/logout, clears token
	fetchProfile	None	Promise<object>	GET /auth/me
	setAccessToken	token	void	Stores token in TokenStore
	getAccessToken	None	string/null	Gets token from TokenStore
	isAuthenticated	None	boolean	True if token exists

4.2.1.2.16 ApiClient

Class	ApiClient			
Description	Helper class (Axios instance) to make HTTP requests to the Backend. Adds Bearer token from TokenStore to request headers. Handles 401 with token refresh.			
Base Class	None			
Constructor	axios.create({ baseURL, withCredentials, ... })			
Prototype	Axios instance			
Source File	services/api.js			
Namespace	@/services			
Attributes	Name	Type	Description	
Methods	Name	Input	Output	Description

4.2.1.2.17 TokenStore

Class	TokenStore
Description	Responsible for persisting access token in memory (not localStorage) to reduce XSS risk. Publishes changes via subscribe for components to react.

Base Class	None			
Constructor	N/A			
Prototype	Storage module			
Source File	services/tokenStore.js			
Namespace	@/services			
Attributes	**Name**	**Type**	**Description**	
Methods	**Name**	**Input**	**Output**	**Description**

4.2.2 UC-01 User Login

4.2.2.1 Screen Design

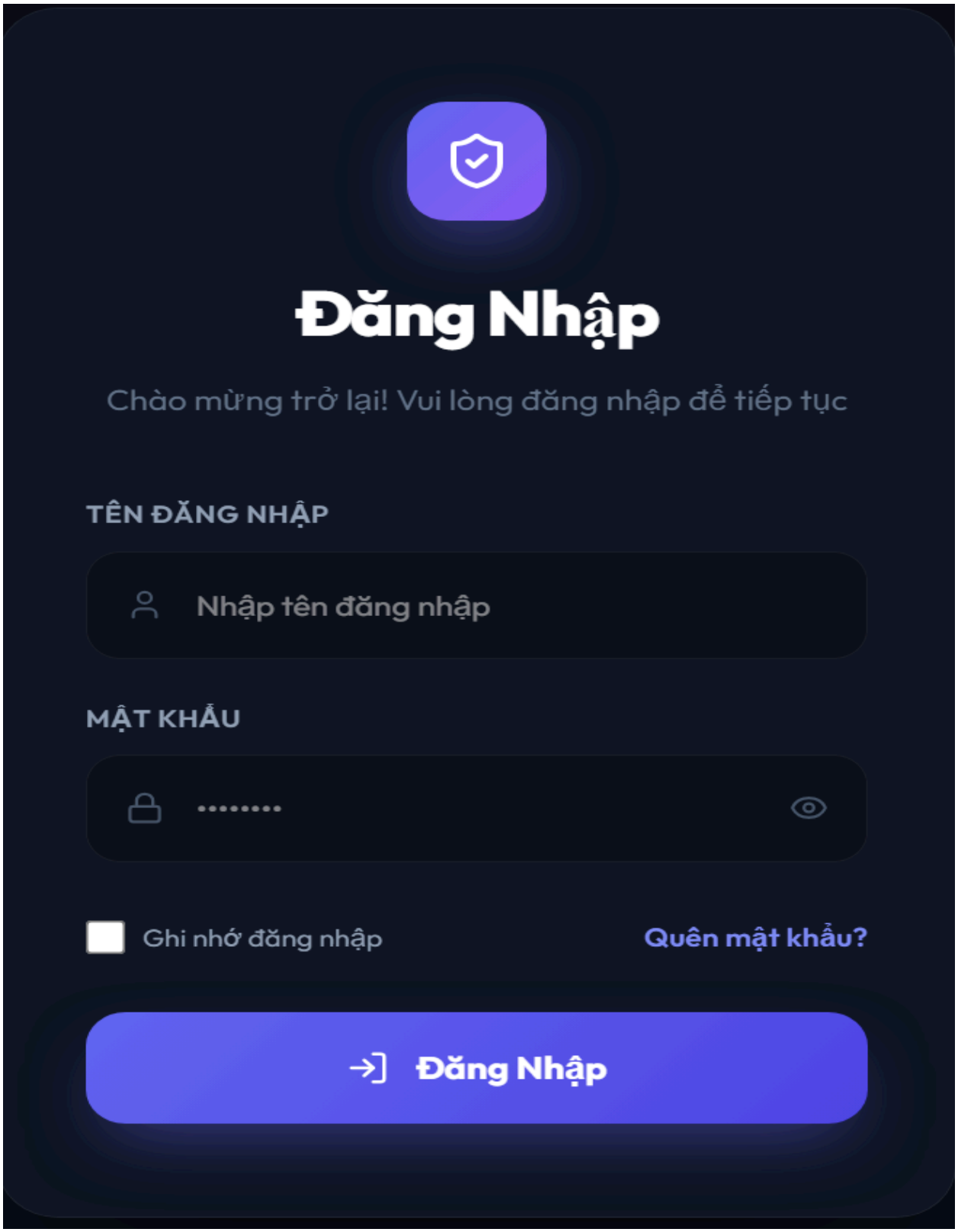
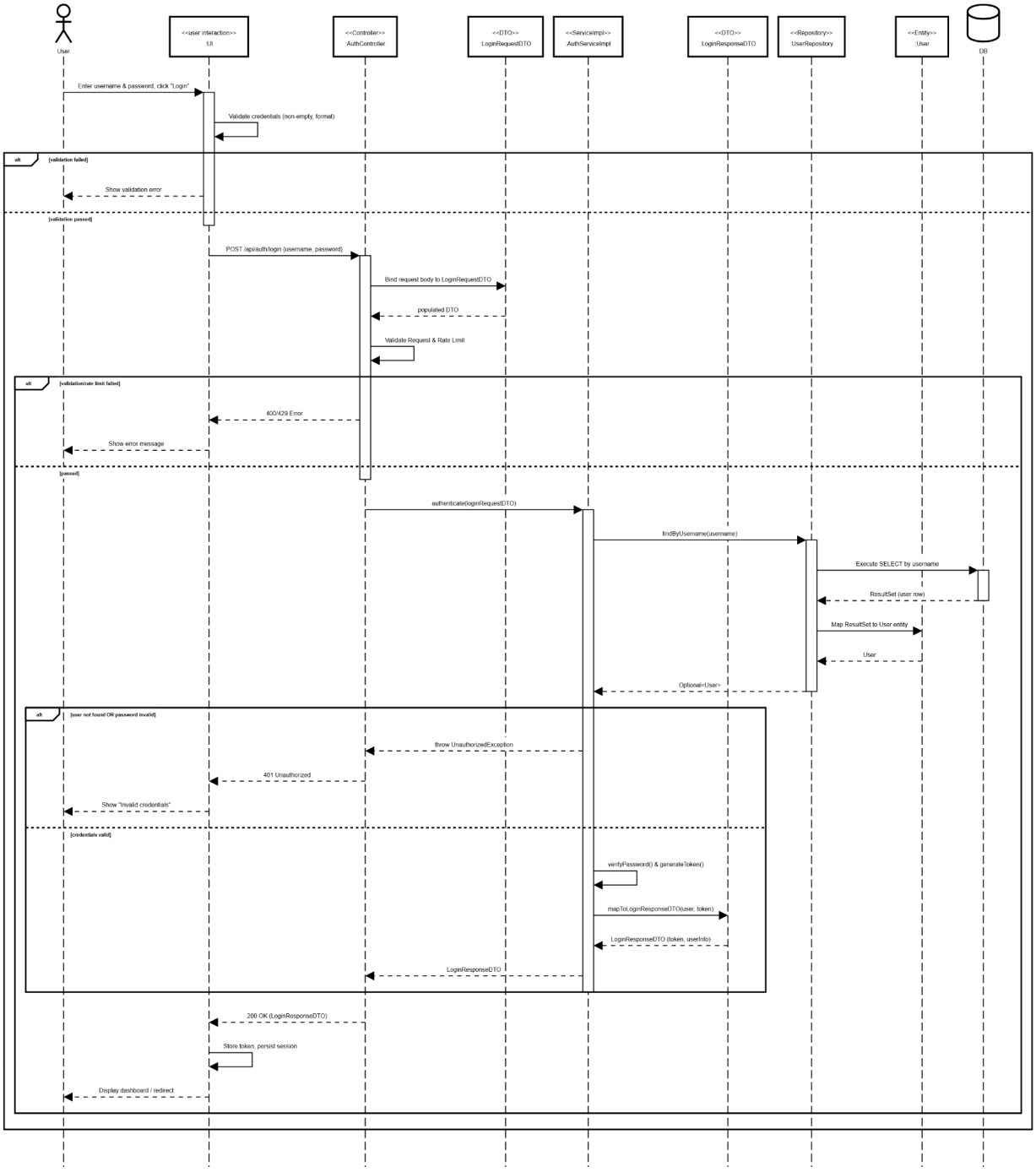


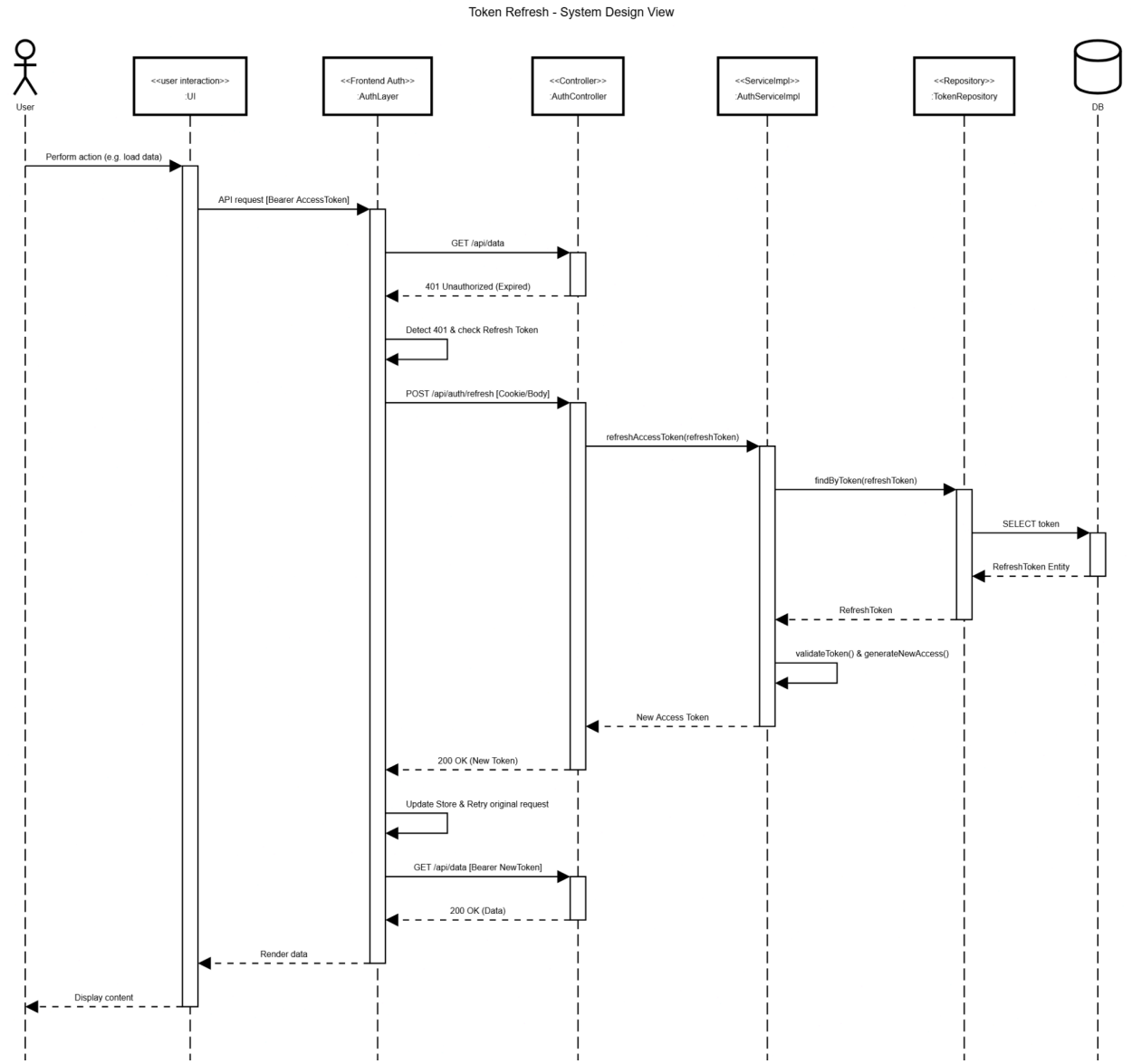
Table 4-1: Screen Definition

No	Object/Control Name	Type	Required	Length	Description
1	username	TextField	Yes	4–20 chars	Input field for username, sent to authentication API.
2	password	PasswordField	Yes	6–50 chars	Secure input field, verified via hashing.
3	rememberMe	Checkbox	No	Default: false	Controls session persistence.
4	forgotPassword	Link	No	Redirect	Navigates to password recovery flow.
5	loginButton	Button	Yes	Trigger API /auth/login	Sends login requests to the backend.

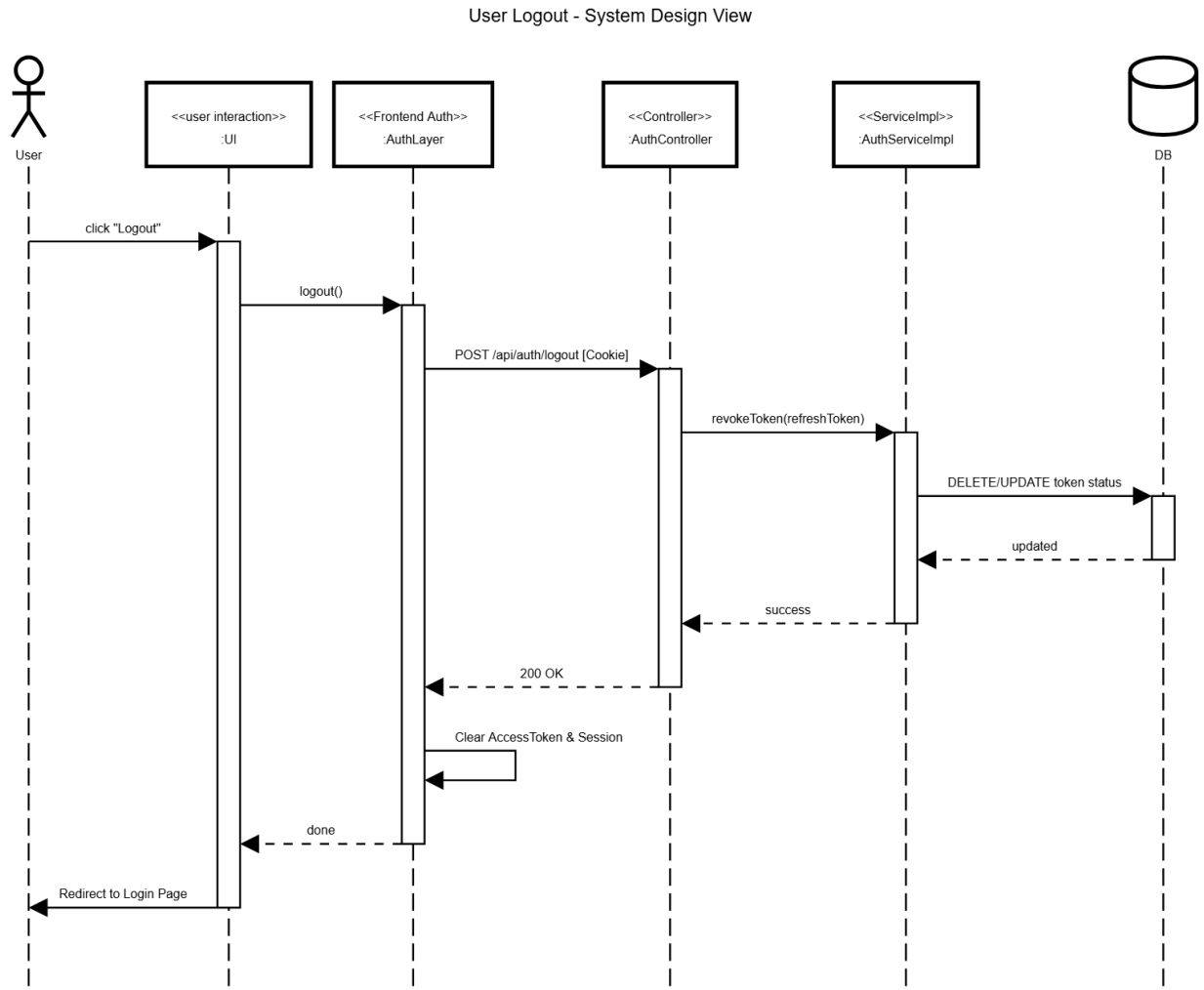
4.2.2.2 Object Interactions Sequence Diagram(s)
Login Flow


User
<<user interaction>>
UI
<<Controller>>
:AuthController
<<DTO>>
:LoginRequestDTO
<<ServiceImpl>>
:AuthServiceImpl
<<DTO>>
:LoginResponseDTO
<<Repository>>
:UserRepository
<<Entity>>
User
DB
Enter username & password, click "Login"
Validate credentials (non-empty, format)
alt
[validation failed]
Show validation error
[validation passed]
POST /api/auth/login (username, password)
Bind request body to LoginRequestDTO
populated DTO
Validate Request & Rate Limit
alt
[validation/rate limit failed]
400/429 Error
Show error message
[passed]
authenticate(loginRequestDTO)
findByUsername(username)
Execute SELECT by username
ResultSet (user row)
Map ResultSet to User entity
User
Optional<User>
alt
[user not found OR password invalid]
throw UnauthorizedException
401 Unauthorized
Show "Invalid credentials"
[credentials valid]
verifyPassword() & generateToken()
mapToLoginResponseDTO(user, token)
LoginResponseDTO (token, userInfo)
LoginResponseDTO
200 OK (LoginResponseDTO)
Store token, persist session
Display dashboard / redirect

Refresh Token



Logout



4.2.3 UC-02 Forgot Password

4.2.3.1 Screen Design

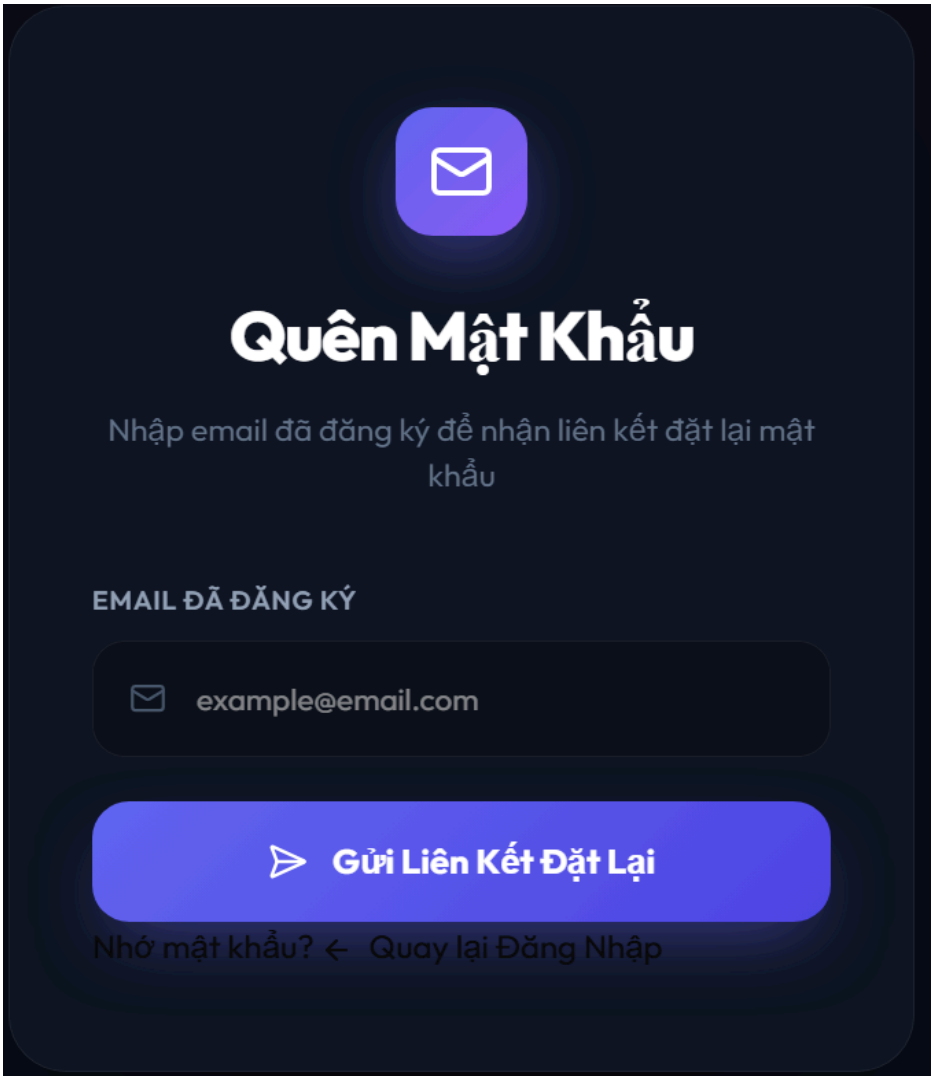
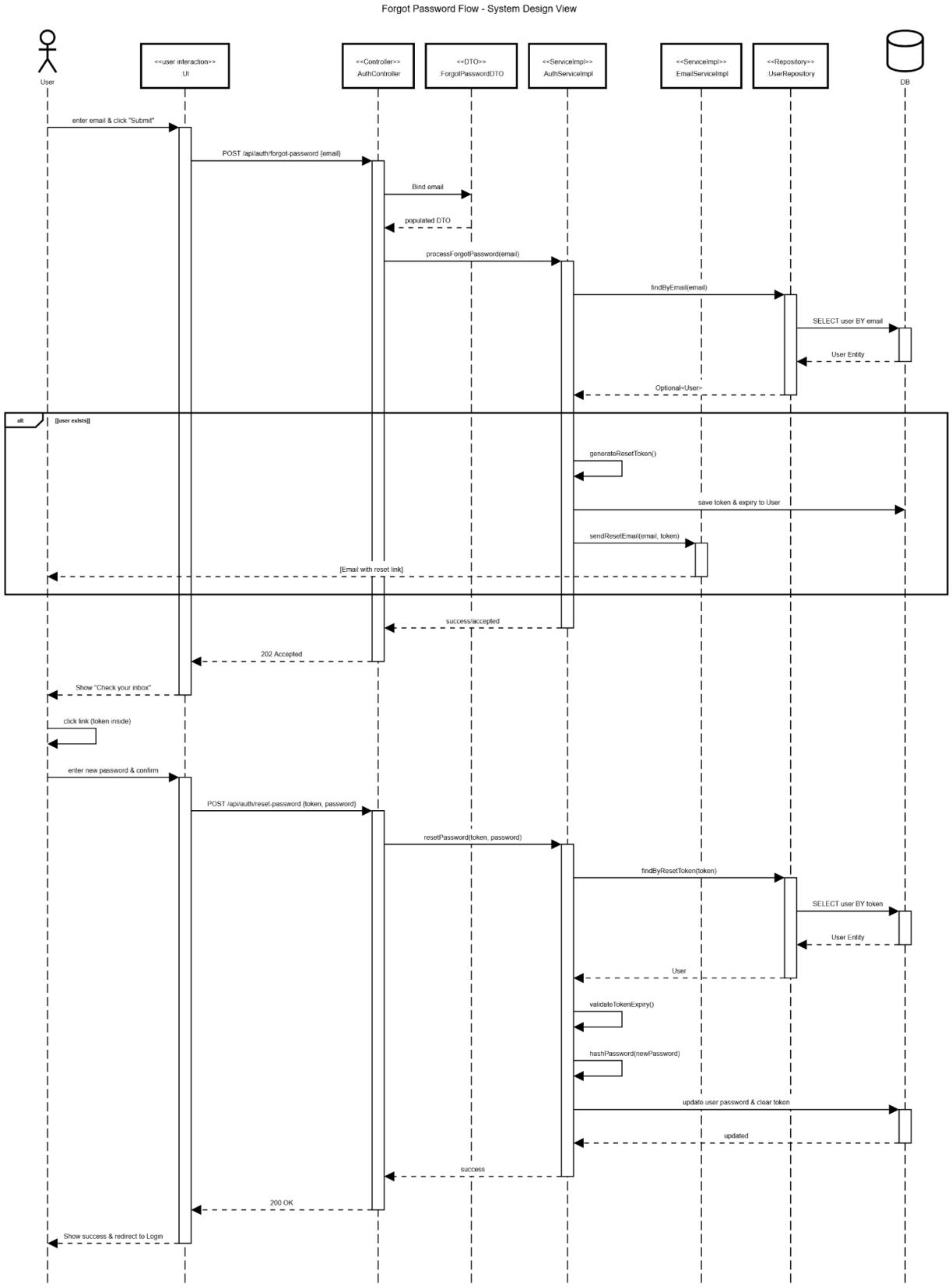


Table 4-2: Screen Definition

No	Object/Control Name	Type	Required	Length	Description
1	email	TextField	Yes	Valid email format	Input field for registered email address.
2	sendResetLinkButton	Button	Yes	Trigger API /auth/forgot-password	Sends password reset request to backend.
3	backToLogin	Link	No	Redirect	Navigates the user back to the login screen.

4.2.3.2 Object Interactions Sequence Diagram(s)



4.2.4 UC-03 User Profile

4.2.4.1 Screen Design

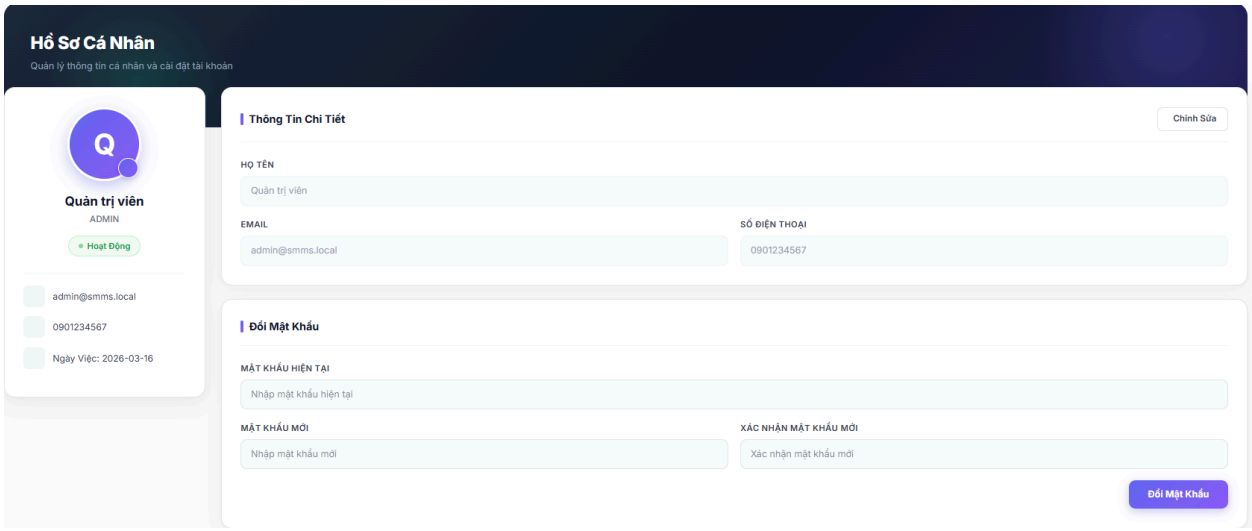


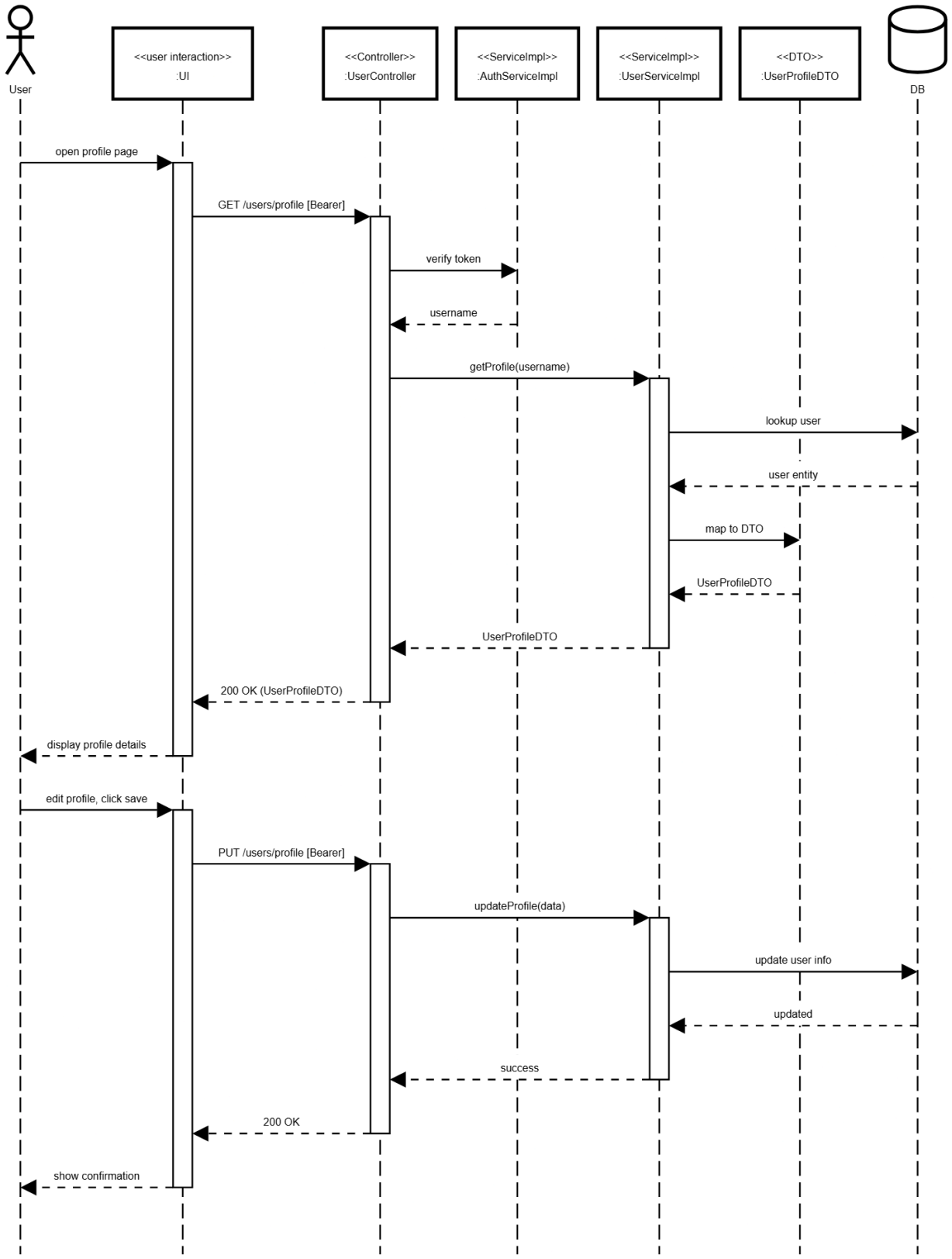
Table 4-3: Screen Definition

No	Object/Control Name	Type	Required	Length	Description
1	pageTitle	Label	Yes	—	Displays the page title "Hồ Sơ Cá Nhân" in the header banner.
2	pageSubtitle	Label	No	—	Displays subtitle text "Quản lý thông tin cá nhân và cài đặt tài khoản" below the page title.
3	avatarImage	Avatar	Yes	—	Displays the user's profile picture or initials as a circular avatar with a purple background.
4	onlineStatusIndicator	Badge	Yes	—	Shows a small circle indicator on the avatar representing the user's online presence.
5	displayName	Label	Yes	Max 50 chars	Displays the full name of the logged-in user below the avatar.
6	roleLabel	Label	Yes	—	Displays the user's role beneath the display name.
7	statusBadge	Badge	Yes	—	Shows the current activity status of the user with a green dot indicator.

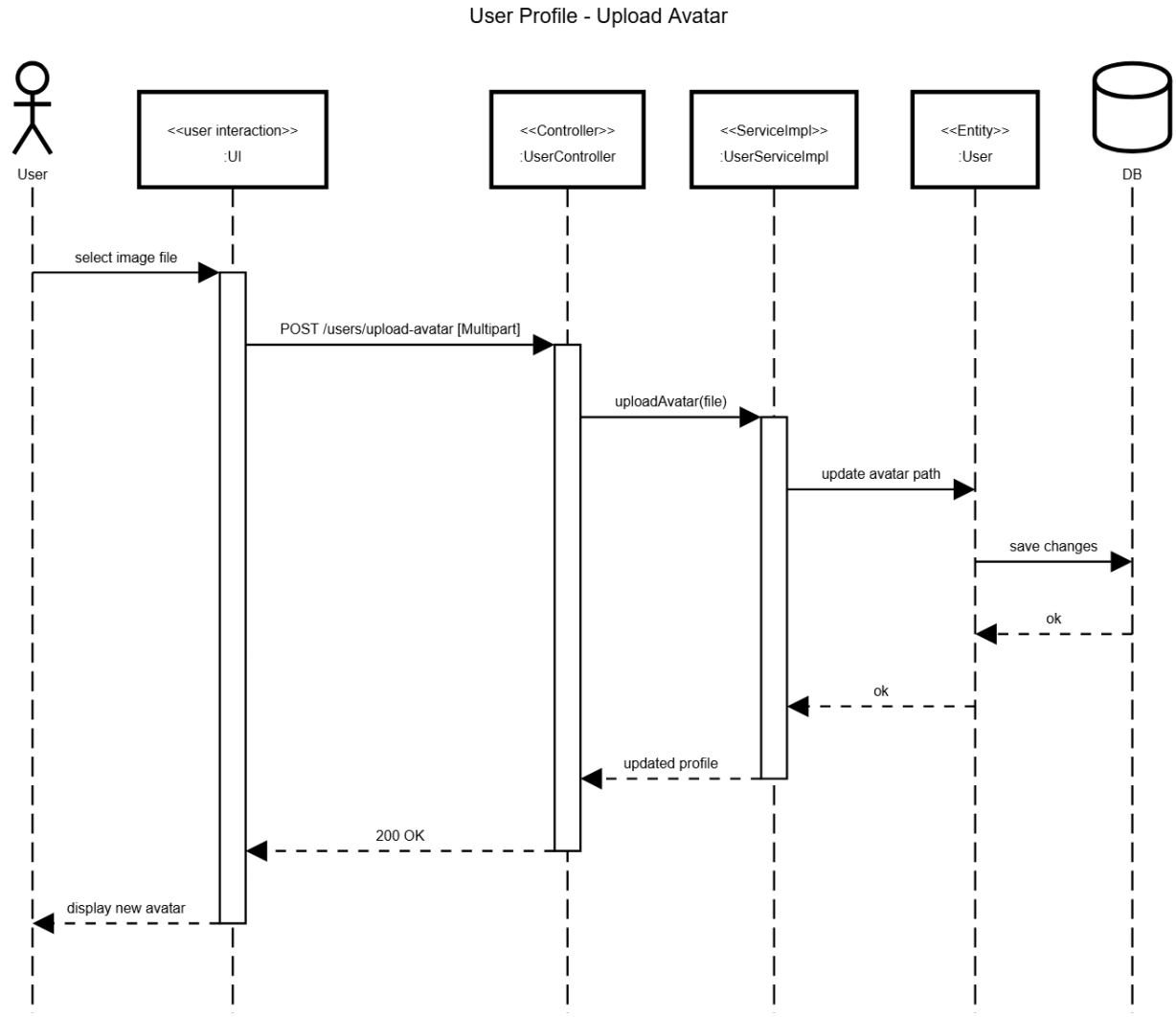
8	sidebarEmail	Label	Yes		Displays the user's email address in the left sidebar info section.
9	sidebarPhone	Label	No	Max 15 chars	Displays the user's phone number in the left sidebar info section.
10	sidebarWorkDate	Label	No	Date format	Displays the user's current work date in the left sidebar.
11	detailSectionTitle	Label	Yes	—	Section header label "Thông Tin Chi Tiết"
12	changeButton	Button	Yes	—	Triggers edit mode for the personal information form, allowing the user to modify their details.
13	nameField	TextField	Yes	Max 100 chars	Input field displaying the user's full name . Read-only unless edit mode is active.
14	emailField	TextField	Yes	Valid email format	Input field displaying the user's registered email address. Read-only unless edit mode is active.
15	phoneField	TextField	No	Max 15 chars	Input field displaying the user's phone number. Read-only unless edit mode is active.
16	upcomingShiftSection Title	Label	Yes	—	Section header label "Ca Làm Việc Sắp Tới"
17	shiftTypeBadge	Badge	Yes	—	Displays the shift type label as a colored badge on the upcoming shift card.
18	shiftDate	Label	Yes	Date format	Displays the date of the upcoming work shift .
19	shiftTimeRange	Label	Yes	Time format	Displays the time range of the upcoming shift.

4.2.4.2 Object Interactions Sequence Diagram(s)
Get & Update Profile

User Profile - Get & Update Info



Avatar Upload



Change Password

4.2.5 UC-04 Change Password

4.2.5.1 Screen Design

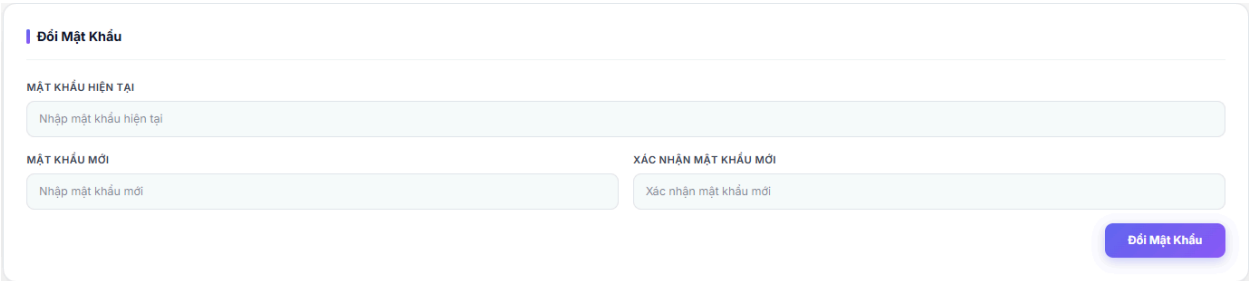
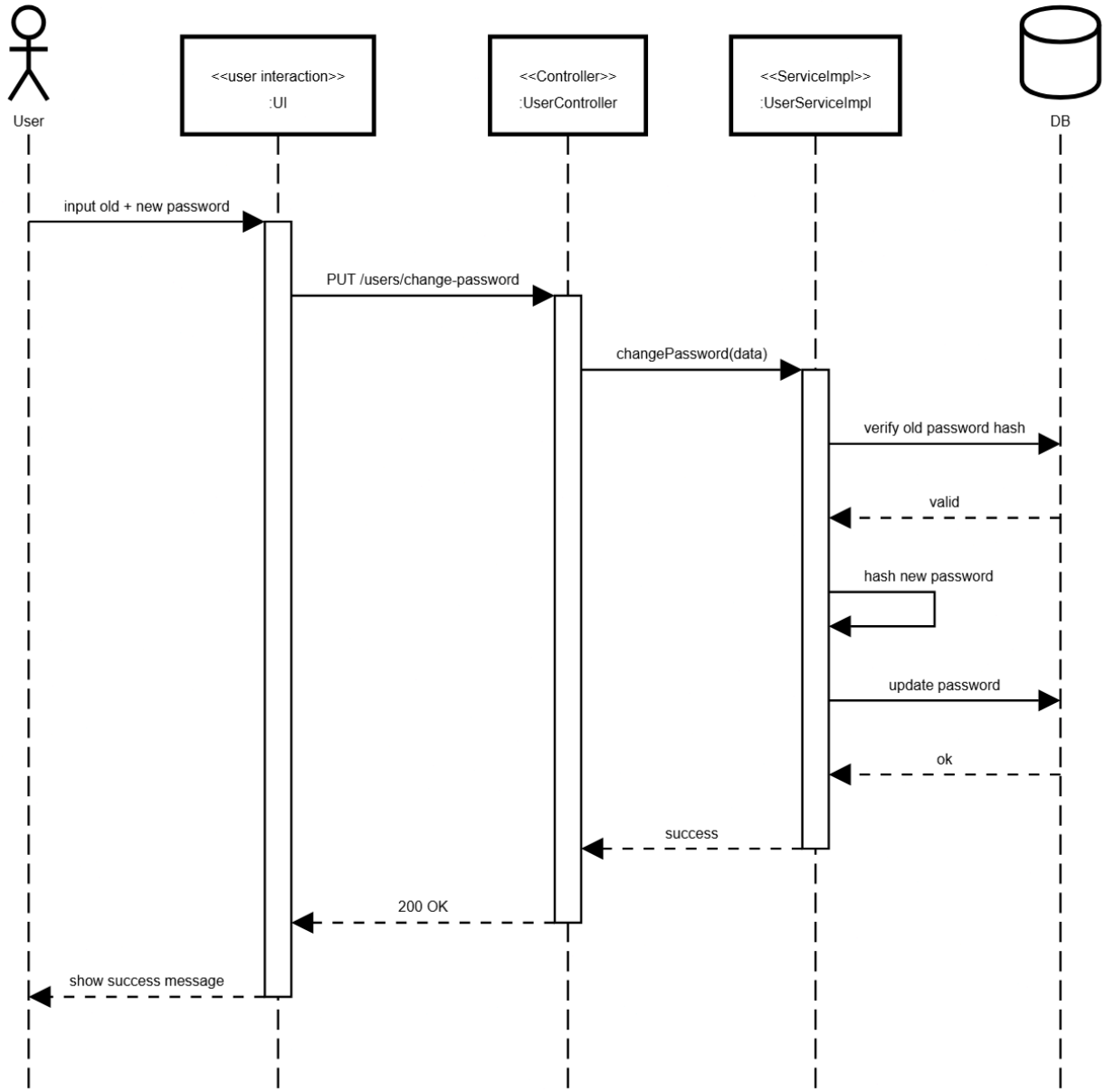


Table 4-4: Screen Definition

No	Object/Control Name	Type	Required	Length	Description
1	sectionTitle	Label	Yes	—	Displays the section header "Đổi Mật Khẩu" with a left blue accent border.
2	currentPasswordField	TextField (Password)	Yes	Max 255 chars	Input field for the user's current password.
3	newPasswordField	TextField (Password)	Yes	Min 8 chars, Max 255 chars	Input field for the user's new password.
4	confirmNewPasswordField	TextField (Password)	Yes	Min 8 chars, Max 255 chars	Input field for confirming the new password.
5	changePasswordButton	Button	Yes	—	Submits the password change form. Triggers API call to update the user's password.

4.2.5.2 Object Interactions Sequence Diagram(s)

User Profile - Change Password



5. Database Design

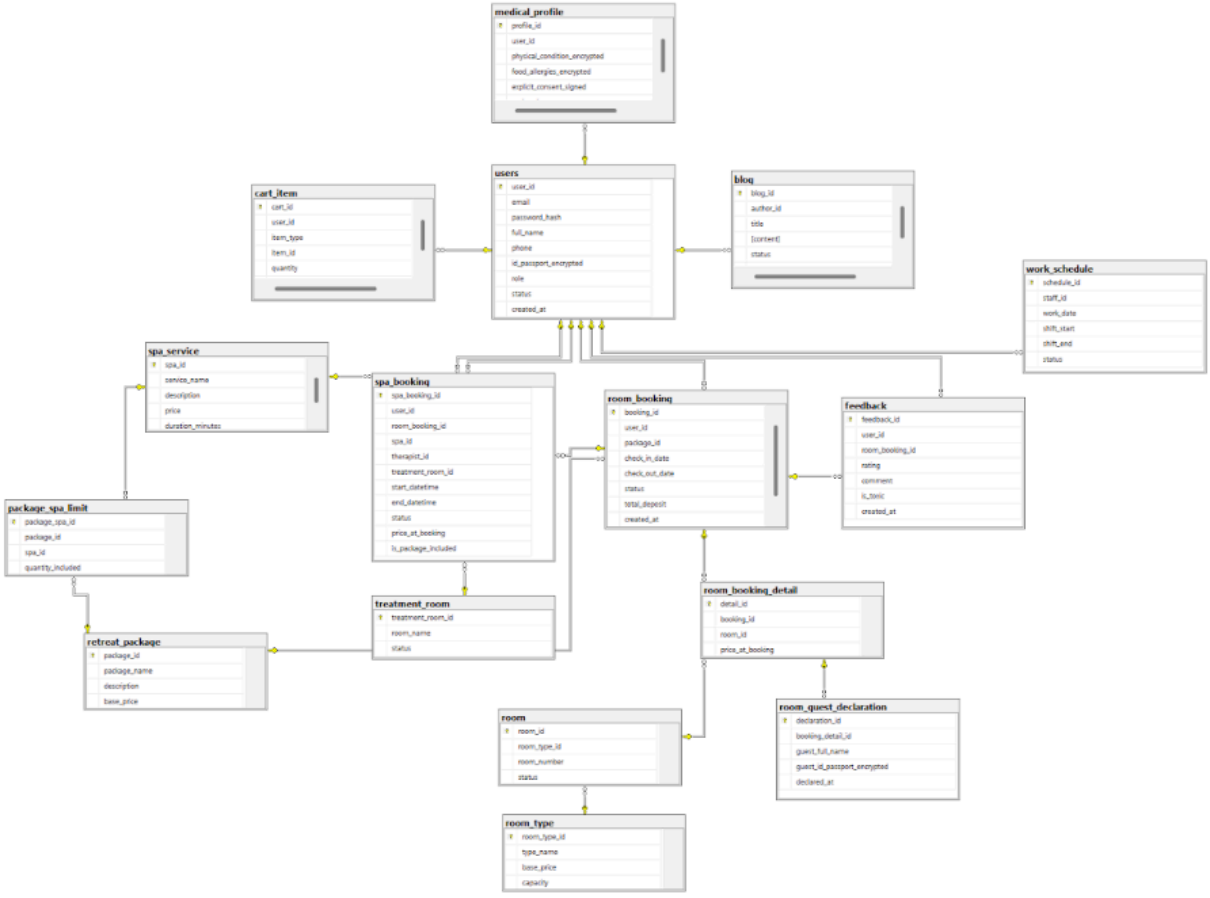


Image detail

5.1 Data File Design

5.1.1.User

#	Attribute name	PK	Type	Mandatory	Description
1	user_id	x	Integer	Yes	Unique internal identifier generated for every user account.
2	email		String	Yes	Account login email address; prevents duplication across profiles.
3	password_hash		String	Yes	One-way salted cryptographic hash of the account password string.
4	full_name		String	Yes	Complete legal name of the user profile holder.

#	Attribute name	PK	Type	Mandatory	Description
5	phone		String	Yes	Contact telephone number for booking notifications and SMS.
6	id_passport_encrypted		String	Yes	AES-256 encrypted National ID/Passport string for police check-in compliance.
7	role		Enum_role	Yes	System permissions category: MANAGER, RECEPTIONIST, THERAPIST, CHEF, CUSTOMER.
8	status		Enum_status	Yes	Profile status controller: ACTIVE (Allowed to log in) or INACTIVE (Account suspended).
9	created_at		datetime	Yes	Historical timestamp tracking when the registration form was initialized.

5.1.2.MEDICAL_PROFILE

#	Attribute name	PK	Type	Mandatory	Description
1	profile_id	x	Integer	Yes	Primary key identifying a specific guest wellness ledger entry.
2	user_id		Integer	Yes	Maps directly to the guest record. Enforces a strict 1-1 relational constraint.
3	physical_condition_encrypted		Text	No	Encrypted string containing physiological health tracking and active chronic diseases.
4	food_allergies_encrypted		Text	No	Encrypted database field log tracking hazardous ingredient alerts for the kitchen.
5	explicit_consent_signed		Boolean	Yes	Mandatory legal authorization flag indicating explicit agreement to data handling rules.
6	updated_at		Datetime	Yes	Captures when a medical profile was last updated or reviewed by staff.

5.1.3.WORK_SCHEDULE

#	Attribute name	PK	Type	Mandatory	Description
1	schedule_id	x	Integer	Yes	Unique identity tag for a specific shift block.
2	staff_id		Integer	Yes	Links to a user record where the role matches a staff type.

3	work_date		Time	Yes	The specific calendar day allocated for this shift line entry.
4	shift_start		Time	Yes	Clock execution time marking the beginning of active working duty.
5	shift_end		Time	Yes	Clock execution time marking the final conclusion of active working duty.
66	status		Enum	Yes	Labor metrics allocation flags

5.1.4.RETREAT_PACKAGE

#	Attribute name	PK	Type	Mandatory	Description
1	package_id	x	Integer	Yes	Unique structural product identifier for the custom combo item.
2	package_name		String	Yes	Public branding name of the retreat program (e.g., "5-day Detox Journey").
3	description		Text	No	Exhaustive breakdown of treatment philosophies, items included, and benefits.
4	base_price		Decimal(15,2)	Yes	Financial base price for package registration, excluding extra options.
5	duration_days		Integer	Yes	Numerical count of total calendar days the retreat covers.

5.1.5.PACKAGE_SPA_LIMIT

#	Attribute name	PK	Type	Mandatory	Description
1	package_spa_id	x	Integer	Yes	Identification index for the package-spa relationship configuration row.
2	package_id		Integer	No	Links configuration parameters to the master package identity.
3	spa_id		Integer	Yes	Links configuration parameters to a distinct target spa operation.
4	quantity_included		Integer	Yes	Total complimentary structural service usage counts allowed within the program.

5.1.6.PACKAGE_FOOD_LIMIT

#	Attribute name	PK	Type	Mandatory	Description

1	package_food_id	x	Integer	Yes	Identification index for the package-food relationship configuration row.
2	package_id		Integer	Yes	Links structural parameters to the parent combo package identity.
3	food_id		Integer	Yes	Links structural parameters to a distinct item on the food menu.
4	quantity_per_day		Integer	Yes	Maximum daily complimentary count threshold allowed per guest.

5.1.7.ROOM_TYPE

#	Attribute name	PK	Type	Mandatory	Description
1	room_type_id	x	Integer	Yes	Master inventory category identity key.
2	type_name		String	Yes	Commercial room label nomenclature (e.g., "Villa 1BR", "Presidential Suite").
3	base_price		Decimal	No	Standard nightly rate applied when a room is booked separate from a package.
4	capacity		String	No	Maximum safe guest occupancy ceiling enforced during booking workflows.

5.1.8.ROOM

#	Attribute name	PK	Type	Mandatory	Description
1	room_id	x	Integer	Yes	Unique internal identifier generated for a physical room unit.
2	room_type_id		Integer	Yes	Relates the physical room asset to its functional pricing tier.
3	room_number		String	Yes	Distinct physical room identifier (e.g., "Villa-101"); prevents duplicate numbers.
4	status		Enum	Yes	Real-time operations flag

5.1.9.ROOM_BOOKING

#	Attribute name	PK	Type	Mandatory	Description
1	booking_id	x	Integer	Yes	Master booking transaction identity key.

2	user_id		Integer	Yes	Identifies the primary customer profile executing the reservation.
3	package_id		Integer	Yes	References the chosen combo package; set to NULL for room-only bookings.
4	check_in_date		DateTime	Yes	Confirmed reservation timestamp marking the start of the resort stay.
5	check_out_date		DateTime	Yes	Confirmed reservation timestamp marking the end of the resort stay.
6	status		Enum	Yes	Workflow tracking flag
7	total_deposit		Decimal	Yes	Down-payment amount processed online to secure the reservation block.
8	created_at		DateTime	Yes	Logs the date and time when the reservation transaction was first created.

5.1.10.ROOM_BOOKING_DETAIL

#	Attribute name	PK	Type	Mandatory	Description
1	detail_id	x	Integer	Yes	Unique primary key identifying a room reservation row.
2	booking_id		String	No	Connects the line item directly to its parent reservation record.
3	room_id		String	Yes	Connects the reservation line item to a specific physical room asset.
4	price_at_booking		String	No	Captures and locks the active nightly room rate at the time of reservation.

5.1.11.ROOM_GUEST_DECLARATION

#	Attribute name	PK	Type	Mandatory	Description
1	declaration_id	x	Integer	Yes	Unique primary index mapping a companion declaration profile row.
2	booking_detail id		Integer	Yes	Links the occupant data directly to a specific reserved physical room.
3	guest_full_nam e		String	Yes	Complete legal name of the accompanying companion occupant.
4	guest_id_passp ort_encrypted		Text	Yes	Encrypted ID/Passport field data used for local police department reports.
	declared_at		DateTime	Yes	Logs the date and time when the guest registration profile was recorded.

5.1.12.SPA_SERVICE

#	Attribute name	PK	Type	Mandatory	Description
1	spa_id	x	Integer	Yes	Unique identity key generated for a specific treatment type.
2	service_name		String	Yes	Name of the spa service treatment (e.g., "Deep Tissue Ayurvedic Massage").
3	description		Text	Yes	Detailed explanation of therapeutic techniques and oils used.
4	price		Decimal	Yes	Standard a la carte rate billed when the service is purchased outside a package.
	duration_minut es		Integer	Yes	Length of the session block in minutes; used to calculate therapist availability.

5.1.13.TREATMENT_ROOM

#	Attribute name	PK	Type	Mandatory	Description

1	treatment_room_id	x	Integer	Yes	Unique primary key identifying a physical spa treatment space.
2	room_name		String	Yes	Physical name or number of the spa room asset (e.g., "Therapy Room A").
3	status		Enum	Yes	Current status indicator for the room

5.1.14.SPA_BOOKING

#	Attribute name	PK	Type	Mandatory	Description
1	spa_booking_id	x	Integer	Yes	Unique primary key identifying an individual spa appointment.
2	user_id		Integer	Yes	Identifies the specific customer profile attending the treatment session.
3	room_booking_id		Integer	No	Connects the appointment to a room folio; set to NULL for walk-in clients.
4	spa_id		Integer	Yes	Identifies the chosen treatment option from the spa service catalog.
5	therapist_id		Integer	Yes	Links to a user record with a role type matching.
6	treatment_room_id		Integer	Yes	Identifies the specific spa room asset reserved for the session.
7	start_datetime		DateTime	Yes	Scheduled start time for the treatment session.
8	end_datetime		DateTime	Yes	Calculated session end time, based on session start + service duration.
9	status		Enum	Yes	Appointment status tracker:

10	price_at_booking		Decimal	Yes	Captures and locks the active service price rate at the time of booking.
11	is_package_included		Boolean	Yes	Flag checking if the session cost is offset by an active retreat package.

5.1.15.CART_ITEM

#	Attribute name	PK	Type	Mandatory	Description
1	cart_id	x	Integer	Yes	Unique key identifying a specific line item row inside the cart.
2	user_id		Integer	Yes	Links the virtual shopping cart row directly to its owner profile.
3	item_type		Enum	Yes	Categorizes the item type
4	item_id		Integer	Yes	Target entity index number pointing to either
5	quantity		Integer	Yes	Number of units requested for purchase within the transaction line.
6	created_at		DateTime	Yes	Tracks when the product row entry was added to the active user cart session.

5.1.16.FOOD_MENU

#	Attribute name	PK	Type	Mandatory	Description
1	food_id	x	Integer	Yes	Master food menu product item index identity key.
2	dish_name		String	Yes	Name of the culinary dish (e.g., "Organic Avocado Quinoa Salad").
3	description		Text	Yes	Complete breakdown of ingredients and macro-nutrient distributions.
4	price		Decimal	Yes	Standard retail meal rate billed outside package allowances.
6	dietary_tags		String	Yes	Group tags matching medical parameters (e.g., "Vegan, Keto, Diabetic-friendly").

5.1.17.FOOD_ORDER

#	Attribute name	PK	Type	Mandatory	Description
1	order_id	x	Integer	Yes	Master meal order transaction header identity key.
2	user_id		Integer	Yes	Identifies the customer profile placing the food order.
3	room_booking_id		Integer	No	Connects the order to a room folio; set to NULL for non-resident walk-in diners.
4	order_time		DateTime	Yes	Logs the date and time when the kitchen order ticket was received.
5	status		Enum	Yes	Kitchen workflow status flags
6	total_amount		Decimal	Yes	Total financial liability balance calculated for all listed order line items.

5.1.18.FOOD_ORDER_DETAIL

#	Attribute name	PK	Type	Mandatory	Description
1	order_detail_id	x	Integer	Yes	Unique primary key identifying a food order item row.

2	order_id		Integer	Yes	Links the item row to its parent master kitchen order ticket.
3	food_id		Integer	Yes	Connects the order line to a specific dish on the menu.
4	quantity		Integer	Yes	Total item units requested for preparation by the kitchen staff.
5	price_at_order		Decimal	Yes	Captures and locks the active menu item price at the time of ordering.
6	special_note		String	No	Custom customer cooking preparation warnings or substitution requests.
7	is_package_included		Boolean	Yes	Flag checking if the item cost is offset by an active retreat package.

5.1.19.INVOICE

#	Attribute name	PK	Type	Mandatory	Description
1	invoice_id	x	Integer	Yes	Central fiscal ledger transaction identity key.
2	user_id		Integer	Yes	Identifies the customer profile responsible for settling the invoice balance.
3	room_booking_id		Integer	Yes	Links the ledger back to its core accommodation or package reservation.
4	room_subtotal		Decimal	Yes	Accumulated costs for lodging options and package bases.
5	spa_subtotal		Decimal	Yes	Sum total of a la carte spa bookings where is FALSE.
6	food_subtotal		Decimal	Yes	Sum total of a la carte food orders where is FALSE.
7	tax_and_fees		Decimal	Yes	Calculated state VAT levies and service fee add-ons.
8	final_amount		Decimal	Yes	Final net cash balance due for payment, computed as subtotals + taxes.
9	status		Enum	Yes	Ledger status tracking flags
10	vnpay_tran_id		String	No	External reference ID string returned by the VNPay gateway.
11	payment_time		DateTime	No	Logs the date and time when payment settlement was verified.

5.1.20.BLOG

#	Attribute name	PK	Type	Mandatory	Description
1	blog_id	x	Integer	Yes	Unique primary key identifying a specific blog article entry.
2	author_id		Integer	Yes	Links to a user record with a role type matching or staff.
3	title		String	Yes	Public headline string displayed for the media entry.
4	content		Text	Yes	Raw body copy and textual payload for the article.
5	status		Enum	Yes	Content visibility state flags.
6	created_at		DateTime	Yes	Logs when the document database row entry was initialized.

5.1.21.FEEDBACK

#	Attribute name	PK	Type	Mandatory	Description
1	feedback_id	x	Integer	Yes	Unique index tag mapping an individual customer review row.
2	user_id		Integer	Yes	Identifies the customer profile submitting the review transaction.
3	room_booking_id		Integer	Yes	Connects the review to a verified stay history record.
4	rating		Integer	Yes	Numeric quality satisfaction score, constrained on a scale from 1 to 5 stars.
5	comment		Text	No	Text review detailing the guest's experience at the resort.
6	is_toxic		Boolean	Yes	Flag checking if automated validation flagged toxic or inappropriate language.
7	created_at		DateTime	Yes	Logs the date and time when the feedback review entry was submitted.